

Pricing, Greeks, and Jump-Risk Management for BTC/ETH Vanilla Options

IEDA4331 Financial Engineering Group Project

Choi Man Hou
mhchoiaf@connect.ust.hk
SID: 20894196

Lee Cheuk Yiu
cyleebt@connect.ust.hk
SID: 21051040

Market data snapshot: 2026-05-19 19:26 UTC

Abstract

This report prices BTC and ETH vanilla options using the core IEDA4331 toolkit: risk-neutral valuation, Black-Scholes pricing, CRR binomial trees, implied volatility, Greeks, and dynamic hedging. We treat the Merton jump-diffusion model as a classical jump benchmark rather than the final innovation. The modern exploration layer then evaluates four crypto-relevant candidates at the same empirical level: stochastic volatility with correlated jumps, market-regime implied stochastic volatility, time-varying jump intensity, and GARCH option pricing. The calibrated Merton benchmark reduces mean absolute pricing error by 80.0% for BTC and 58.2% for ETH relative to a historical-volatility Black-Scholes baseline. The refreshed equal-level comparison is deliberately nuanced: **Merton gives the lowest BTC MAE, MR-ISVM gives the lowest ETH MAE, SVCJ produces the largest standardized crash-protection premium**, and hedge-error leadership depends on the asset and risk objective. The main conclusion is that crypto option markets price non-Gaussian jump and volatility-regime risk explicitly; Black-Scholes remains a necessary no-arbitrage benchmark, but it is not sufficient as a standalone crypto risk engine.

Contents

1	Introduction and Course Motivation	3
2	Data and Market Snapshot	3
3	Evaluation Metrics and Comparative Protocol	4
4	Baseline Pricing Framework	5
4.1	Black-Scholes Benchmark	5
4.2	CRR Binomial Tree	5
5	Candidate Model Formulations	6
5.1	Merton as Classical Jump Benchmark	6
5.2	Candidate A: Stochastic Volatility with Correlated Jumps	7
5.3	Candidate B: Market-Regime Implied Stochastic Volatility	7
5.4	Candidate C: Time-Varying Jump Intensity	8
5.5	Candidate D: GARCH Option Pricing	8
6	Historical Distribution and Jump Diagnostics	9
7	Market Calibration and Statistical Evidence	10
8	Interest Rate Sensitivity	12
9	Jump-Parameter Sensitivity	13
10	Greeks and Risk Management	14
11	Jump-Hedging Simulation	16
12	Discussion	17
13	Conclusion	17
A	Implementation Notes	19
B	Core Programming Code Listings	20
C	AI Assistance Statement	46

1 Introduction and Course Motivation

This report studies BTC and ETH vanilla options under the central language of IEDA4331: no-arbitrage pricing, risk-neutral valuation, binomial replication, the Black-Scholes-Merton PDE, implied volatility, Greeks, and hedging error. The objective is to move from classroom pricing identities to a market-calibrated crypto option risk engine, then ask where the classical diffusion model fails. We use the Black-Scholes model [2, 8], CRR binomial tree [4], and Merton jump-diffusion model [9] as the classical ladder of benchmarks.

The Risk-Neutral Pricing chapter brings the analysis from physical expected returns to martingale valuation. Instead of asking whether BTC or ETH is expected to appreciate, we price contingent claims under Q :

$$V_t = E_t^Q \left[e^{-r(T-t)} V_T \right].$$

The Binomial Tree chapter brings the analysis to finite-state replication: a derivative is priced by constructing a self-financing portfolio whose future payoff matches the claim. The Black-Scholes-Merton chapter then takes the limiting continuous-time view and derives the PDE from a self-financing delta hedge:

$$\frac{\partial V}{\partial t} + rS \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rV = 0.$$

The Risk Management chapter brings the analysis from valuation to control: Delta, Gamma, Vega, discrete hedging error, VaR logic, and stress testing. Our report follows that course arc, then adds a jump process precisely where the diffusion assumption breaks down for cryptocurrency markets.

2 Data and Market Snapshot

We use public Deribit data for BTC and ETH: `get_book_summary_by_currency` for option books, `get_instruments` for contract metadata, and `get_index_chart_data` for one-year index histories. Deribit option premiums are quoted in coin units, so we convert them into USD:

$$\text{Option price}_{USD} = \text{Option price}_{coin} \times S.$$

Table 1: Deribit option universe after cleaning

Currency	Raw rows	Usable rows	Median spot	Median IV	Min days	Max days
BTC	878	688	76975.3	42.13%	1.52	310.5
ETH	710	474	2118.6	55.29%	1.52	310.5

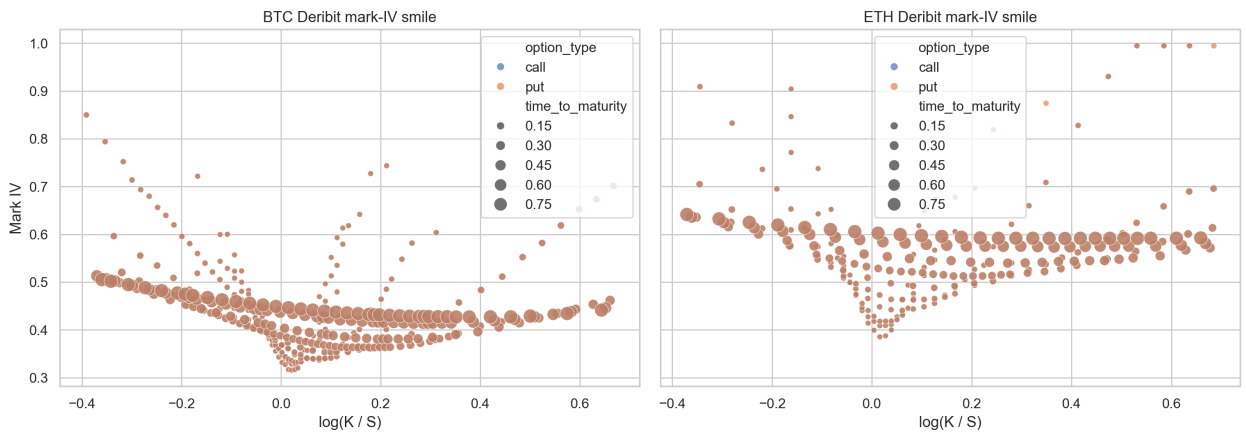


Figure 1: Deribit mark implied-volatility smiles across strikes and maturities.

Inference. A flat Black-Scholes volatility input would imply a roughly horizontal surface. The observed BTC and ETH smiles are curved and dispersed, indicating that the market prices skew, maturity effects, and non-Gaussian tail risk.

The course's implied-volatility concept is important here. Implied volatility is not simply another volatility estimator; it is the volatility that forces the Black-Scholes formula to match a traded option price. A smile therefore means the market is using Black-Scholes as a quote convention while rejecting its constant-volatility distributional assumption. Crypto option studies document precisely this pattern in Bitcoin options [12, 3]. This is the bridge from the data section to the model-innovation section.

Comparative lens. The same smile carries different information for each model. Black-Scholes treats it as model error; Merton and dynamic jump intensity read it as evidence of discontinuous tail risk; MR-ISVM reads it as a state-dependent surface; GARCH reads it as volatility clustering; and SVCJ reads it as joint spot-volatility stress.

3 Evaluation Metrics and Comparative Protocol

To keep the candidate comparison at the same level throughout the report, every model is evaluated on the same cleaned option universe. For option i , let V_i^{mkt} denote the Deribit USD market price and $\hat{V}_i^{(m)}$ denote the price produced by model m . The model pricing error is:

$$e_i^{(m)} = \hat{V}_i^{(m)} - V_i^{mkt}.$$

The key cross-sectional pricing metrics are:

$$\begin{aligned} \text{Bias}^{(m)} &= \frac{1}{N} \sum_{i=1}^N e_i^{(m)}, \\ \text{MSE}^{(m)} &= \frac{1}{N} \sum_{i=1}^N \left(e_i^{(m)} \right)^2, \quad \text{RMSE}^{(m)} = \sqrt{\text{MSE}^{(m)}}, \\ \text{MAE}^{(m)} &= \frac{1}{N} \sum_{i=1}^N \left| e_i^{(m)} \right|, \quad \text{MedAE}^{(m)} = \text{median}_i \left(\left| e_i^{(m)} \right| \right), \\ \text{P90AE}^{(m)} &= Q_{0.90} \left(\left| e_i^{(m)} \right| \right). \end{aligned}$$

MAE is the main headline metric because it is in USD units and is less dominated by a small number of very expensive contracts than MSE. RMSE is still reported because it penalizes large pricing misses more aggressively.

For pairwise improvement against Black-Scholes, we use:

$$\Delta \text{AE}_i^{(m)} = \left| e_i^{BS} \right| - \left| e_i^{(m)} \right|, \quad \text{ShareImproved}^{(m)} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\Delta \text{AE}_i^{(m)} > 0\}.$$

The paired tests in the calibration section test whether $\Delta \text{AE}_i^{(m)}$ is significantly positive for each candidate model.

For standardized crash protection, we price a 30-day 5% out-of-the-money put under each model and define its model premium over Black-Scholes as:

$$\text{CrashPremium}^{(m)} = \hat{V}_{30d, 5\% \text{ OTM put}}^{(m)} - \hat{V}_{30d, 5\% \text{ OTM put}}^{BS}.$$

For hedge simulations, each path p produces a final short-option hedge P&L, denoted $\text{PnL}_p^{(m)}$. We summarize hedging with:

$$\text{MeanAbsHedgeError}^{(m)} = \frac{1}{P} \sum_{p=1}^P \left| \text{PnL}_p^{(m)} \right|, \quad \text{LeftTail}_{5\%}^{(m)} = Q_{0.05} \left(\text{PnL}_p^{(m)} \right).$$

The first metric measures average replication error; the second measures adverse tail loss.

4 Baseline Pricing Framework

4.1 Black-Scholes Benchmark

For European calls and puts with no dividend yield:

$$C = SN(d_1) - Ke^{-rT}N(d_2), \quad P = Ke^{-rT}N(-d_2) - SN(-d_1),$$

where

$$d_1 = \frac{\log(S/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}.$$

Volatility is estimated from log returns, following the course treatment that log returns aggregate naturally across time and that annualized volatility scales with the square root of time. The physical drift is not used directly in risk-neutral pricing because the Black-Scholes PDE and risk-neutral valuation remove μ through replication.

4.2 CRR Binomial Tree

The CRR tree uses

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = \frac{1}{u}, \quad p_Q = \frac{e^{r\Delta t} - d}{u - d}.$$

At each node:

$$V_t = e^{-r\Delta t}(p_Q V_u + (1 - p_Q)V_d).$$

For American options, the backward step replaces continuation value with

$$V_t = \max\{\text{continuation value, immediate exercise value}\}.$$

This matters because the binomial tree is not only a numerical method. It is the discrete version of the course's no-arbitrage argument: p_Q is not the real-world probability that BTC or ETH goes up, but the probability that makes discounted prices consistent with replication.

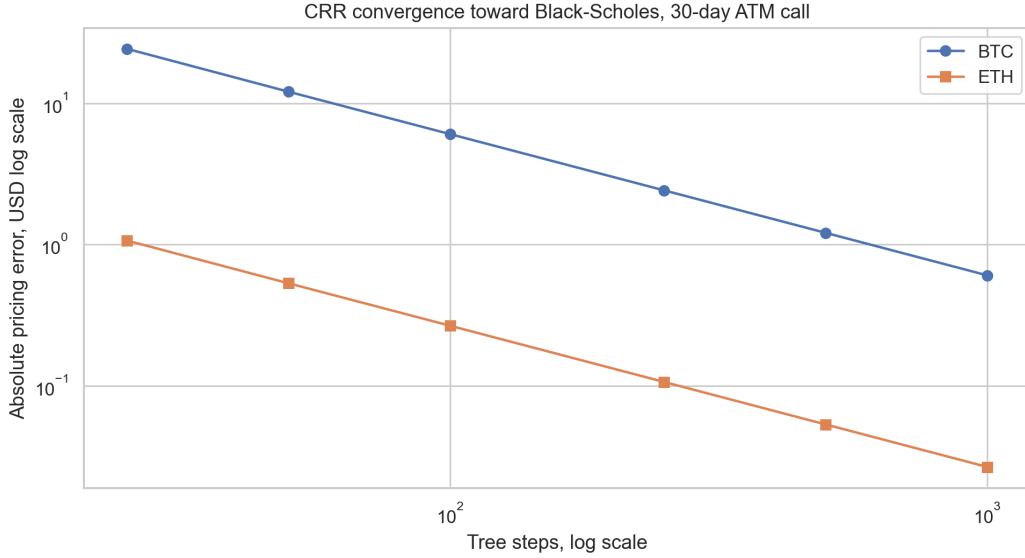


Figure 2: CRR convergence toward the Black-Scholes benchmark for 30-day at-the-money calls.

Table 2: European and American CRR pricing, 30-day at-the-money options

Currency	Option	Spot/Strike	BS Eur.	CRR Eur.	CRR Am.	Am. premium
BTC	call	76975.3	2449.75	2448.54	2448.54	0.00
BTC	put	76975.3	2386.55	2385.34	2388.59	3.25
ETH	call	2118.6	107.37	107.32	107.32	0.00
ETH	put	2118.6	105.64	105.58	105.67	0.09

Inference. The American call premium is effectively zero for non-dividend underlyings, while American puts carry a small early-exercise premium. This is exactly the qualitative result expected from the course theory: early exercise destroys call time value when there is no dividend benefit, but it can be rational for puts when the option is sufficiently in the money.

Comparative role. Black-Scholes and CRR are deliberately kept as the control models. They are not expected to win the crypto smile contest; their role is to anchor no-arbitrage logic, expose how far a diffusion benchmark misses market prices, and make later candidate improvements economically interpretable.

5 Candidate Model Formulations

5.1 Merton as Classical Jump Benchmark

The Merton jump-diffusion model was introduced in 1976 [9]. It is therefore not recent enough to be the final innovation by itself. In this report, it serves as the first serious extension beyond Black-Scholes: a classical benchmark that tests whether explicit price jumps improve crypto option pricing.

The model replaces the diffusion-only assumption with:

$$\frac{dS_t}{S_{t-}} = (r - \lambda\kappa)dt + \sigma dW_t^Q + (J - 1)dN_t,$$

where

$$\log J \sim N(\mu_J, \delta_J^2), \quad \kappa = E[J - 1] = e^{\mu_J + \frac{1}{2}\delta_J^2} - 1.$$

Here λ is the annual jump intensity, μ_J is average log jump size, and δ_J is jump-size volatility. The compensator $\lambda\kappa$ keeps the discounted stock price consistent with risk-neutral valuation.

Conditional on n jumps, terminal log-price remains Gaussian. Hence the Merton option value is a Poisson mixture of conditional Black-Scholes-style prices:

$$V^{Merton}(S, K, T) = \sum_{n=0}^{\infty} e^{-\lambda T} \frac{(\lambda T)^n}{n!} V_n^{BS}(S, K, T).$$

This extension preserves the course's risk-neutral framework while adding a realistic channel for crypto discontinuities. The compensator is essential: without it, the jump term would change the drift of the discounted stock process and break the martingale logic that motivates risk-neutral pricing.

5.2 Candidate A: Stochastic Volatility with Correlated Jumps

The first modern candidate is a stochastic volatility with correlated jumps (SVCJ) model. This model family is closely related to affine jump-diffusion work in option pricing [1, 5], and it has been applied directly to cryptocurrency options in recent empirical work [6]. Its core idea is that crypto crashes are not only spot-price jumps; they are often volatility events as well.

A compact SVCJ specification is:

$$\begin{aligned} \frac{dS_t}{S_{t-}} &= (r - \lambda\mu_S)dt + \sqrt{v_t} dW_t^S + Y_t^S dN_t, \\ dv_t &= \kappa_v(\theta_v - v_t)dt + \xi\sqrt{v_t} dW_t^v + Y_t^v dN_t, \quad dW_t^S dW_t^v = \rho dt. \end{aligned}$$

Here v_t is stochastic variance, ρ captures leverage/skew, and (Y_t^S, Y_t^v) allows price jumps and volatility jumps to arrive together. Compared with Merton, SVCJ can explain a more realistic crypto stress state:

spot falls sharply + implied volatility jumps upward.

In the implemented proxy, the historical return series initializes v_0 , θ_v , κ_v , variance-jump size, and leverage correlation. The option calibration then refines only the parameters that enter the moment-matched pricing equation directly: the variance multiplier and vol-of-vol ξ . This avoids an unidentified calibration problem: ρ is economically important for skew, but in our reduced moment proxy it is reported as a historical state estimate rather than optimized as if it affected prices.

5.3 Candidate B: Market-Regime Implied Stochastic Volatility

The second candidate is a market-regime implied stochastic volatility model (MR-ISVM), motivated by recent crypto option research [10]. The key idea is that crypto option surfaces behave differently across regimes such as calm markets, trending markets, and stress markets. A reduced formulation is:

$$\begin{aligned} z_t &\in \{1, \dots, M\}, \quad \sigma_{IV}(K, T, t) = f_{z_t}(K, T), \\ V_t &= BS(S_t, K, T, r, \sigma_{IV}(K, T, t)). \end{aligned}$$

The regime variable z_t can be estimated from returns, realized volatility, volume, or option-surface features. Compared with Merton, MR-ISVM does not force one set of jump parameters to explain all market states; it allows the implied volatility surface itself to become state-dependent.

Our fitted MR-ISVM surface uses a 12-factor design in log-moneyness $x = \log(K/S)$:

$$\begin{aligned} \sigma_{IV} = \text{clip} &(\beta_0 + \beta_1 x + \beta_2 |x| + \beta_3 x^2 + \beta_4 \sqrt{T} + \beta_5 T \\ &+ \beta_6 I_{put} + \beta_7 I_{put} x + \beta_8 I_{short} \\ &+ \beta_9 x \sqrt{T} + \beta_{10} I_{put} \sqrt{T} + \beta_{11} x^3, \sigma_{\min}(T), \sigma_{\max}). \end{aligned}$$

The cross term $x\sqrt{T}$ lets skew vary by maturity, $I_{put}\sqrt{T}$ lets the put wing have a different term slope, and x^3 adds asymmetric wing curvature. This is the main code-level refinement from the teammate merge: MR-ISVM is no longer just a static nine-term smile fit, but a regime-aware and maturity-aware surface benchmark.

5.4 Candidate C: Time-Varying Jump Intensity

The third candidate is a dynamic jump-intensity model. Recent Bitcoin option evidence emphasizes that jump magnitude and jump arrival rates are time-varying [3]. This is important for crypto because large moves can cluster through leverage, liquidations, and margin pressure.

A parsimonious extension is:

$$\frac{dS_t}{S_{t-}} = (r - \lambda_t \kappa) dt + \sigma dW_t^Q + (J - 1) dN_t,$$

$$d\lambda_t = \beta(\bar{\lambda} - \lambda_t) dt + \alpha dN_t.$$

After a jump, λ_t increases, then mean-reverts. Compared with Merton's constant λ , this model captures jump clustering and liquidation-cascade risk.

5.5 Candidate D: GARCH Option Pricing

The fourth candidate is a GARCH option-pricing model, which has also been studied for Bitcoin options [11]. It is a discrete-time alternative that connects naturally to the course's historical volatility estimation section:

$$r_{t+1} = \mu + \sqrt{h_{t+1}} \varepsilon_{t+1},$$

$$h_{t+1} = \omega + \alpha r_t^2 + \beta h_t.$$

Option prices can then be computed by simulation under a risk-neutralized conditional variance process. Compared with Merton, GARCH does not require discontinuous jumps, but it captures volatility clustering. It is therefore useful as a competing explanation for smiles: are crypto smiles mainly jump risk, or time-varying volatility?

Table 3: Equal-level implementation protocol for all candidate models

Model	Calibration object	Pricing engine	Same-level diagnostic
Black-Scholes	Historical log-return variance	Closed-form diffusion formula	Full-universe MAE/RMSE and Greeks
Merton	Weighted Deribit option-price error	Poisson mixture of Black-Scholes prices	Full-universe MAE/RMSE, jump sensitivity, hedging
SVCJ proxy	Historical variance state plus identifiable variance and ξ calibration	Moment-matched stochastic variance with Merton jumps	Full-universe MAE/RMSE and standardized crash put
MR-ISVM	Regime-aware Deribit mark-IV cross section	Black-Scholes with active-regime fitted IV surface	Full-universe MAE/RMSE and standardized crash put
Dynamic jump	Weighted option-price error with mean-reverting λ_t	Merton mixture with horizon-dependent effective intensity	Full-universe MAE/RMSE and standardized crash put
GARCH	Historical conditional return variance	Black-Scholes with forecast average variance	Full-universe MAE/RMSE and standardized crash put

This protocol is deliberately symmetric. Each candidate uses the same cleaned Deribit universe, the same 80-contract calibration discipline where option calibration is needed, and the same diagnostic definitions from the previous section. The SVCJ implementation is labelled as a proxy because a

full affine SVCJ calibration normally requires a time series of option surfaces; here we use a moment-matched variance state so it can be compared fairly with the available live snapshot. The empirical comparison is then carried through the distribution, calibration, sensitivity, Greeks, and hedging sections rather than isolated here.

Table 4: Regime-aware MR-ISVM surface diagnostics

Currency	Active regime	Calibrated regimes	Coefficients	Rel. price RMSE	IV floor/cap
BTC	stress	calm, trend	12	0.3699	0.20 / 3.00
ETH	stress	calm, stress	12	0.2783	0.20 / 3.00

Model-design inference. Both assets are detected in stress regimes in this snapshot. ETH has a directly fitted stress surface, while BTC falls back to fitted calm/trend surfaces because the stress slice is sparse in the live option snapshot. This is exactly why the surface model is useful in a course project: the Black-Scholes formula remains the pricing map, while the implied volatility input becomes a market-state object rather than a single scalar.

6 Historical Distribution and Jump Diagnostics

We compute one-year BTC/ETH log returns from Deribit index history. Jumps are detected using a robust z-score threshold of 4.0. Non-jump observations initialize diffusive volatility; jump observations initialize jump intensity and jump-size distribution.

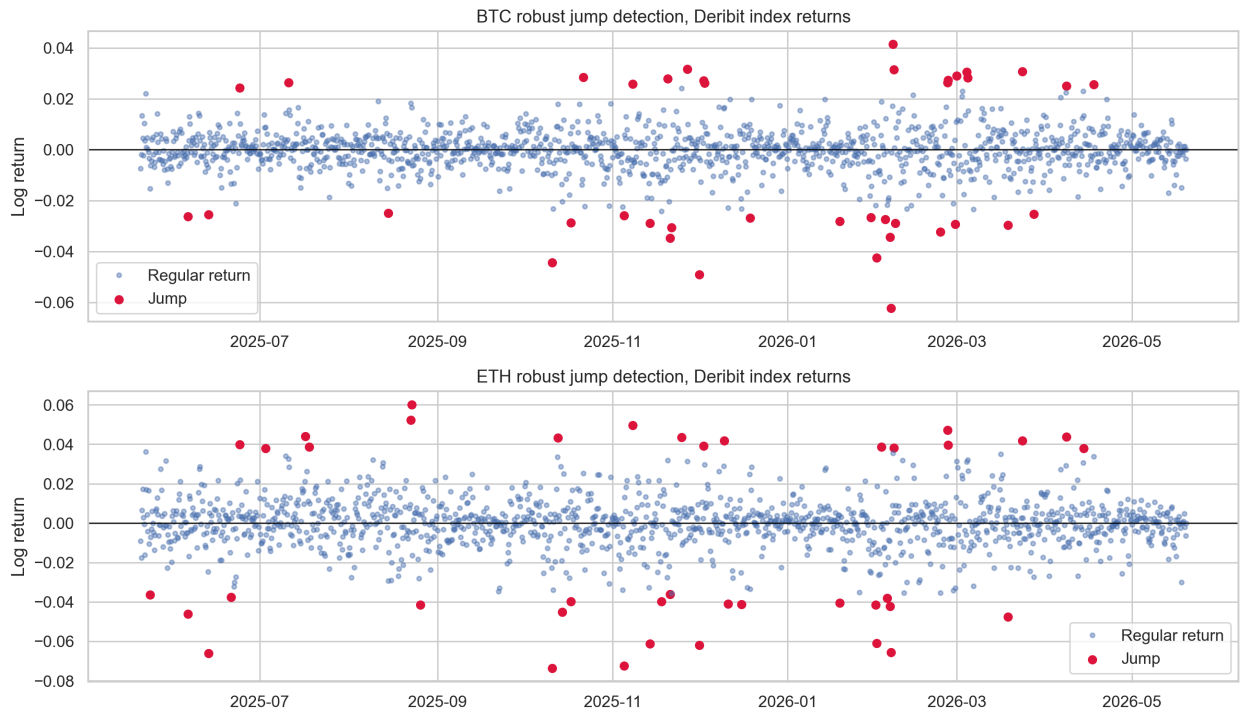


Figure 3: Robust jump detection in BTC and ETH index log returns.

Table 5: Historical jump initialization

Currency	Obs.	Jumps	Diff. σ	λ	μ_J	δ_J	Worst	Best
BTC	1459	40	27.49%	40.05	-0.500%	3.157%	-6.240%	4.139%
ETH	1459	40	44.02%	40.05	-0.748%	4.746%	-7.373%	6.014%

Table 6: Return distribution diagnostics

Currency	Ann. mean	Ann. vol	Skew	Ex. kurtosis	JB statistic	JB p-value
BTC	-32.55%	33.68%	-0.547	4.918	1530.39	$< 10^{-12}$
ETH	-18.98%	52.79%	-0.399	3.623	829.07	$< 10^{-12}$

Inference. Jarque-Bera tests strongly reject normality for both BTC and ETH returns. Both assets exhibit negative skew and excess kurtosis, validating the modeling decision to go beyond a lognormal diffusion. ETH has higher annualized volatility and jump-size volatility, while BTC still exhibits material discontinuous downside risk.

This section deepens the Black-Scholes estimation chapter. In the classical model, the statistic we need is the standard deviation of log returns. For BTC/ETH, that statistic is still useful, but it is incomplete: skewness, kurtosis, and jump frequency explain why a single σ cannot reproduce the market smile. The empirical distribution therefore motivates a pricing model with both a continuous volatility channel and a discontinuous jump channel.

Comparative lens. These diagnostics initialize different model states. Black-Scholes uses only diffusive volatility; Merton uses diffusive volatility plus jump intensity and jump-size moments; dynamic jump intensity asks whether λ_t should vary across horizons; GARCH uses conditional variance persistence; SVCJ uses both variance state and variance-jump state; and MR-ISVM uses the option surface itself to infer the current regime.

7 Market Calibration and Statistical Evidence

Merton parameters are calibrated to liquid Deribit option marks by minimizing weighted relative pricing error. The calibration uses 80 liquid options per asset; diagnostics are computed on the full cleaned universe. Conceptually, this is an implied-parameter exercise: just as the course defines implied volatility by inverting Black-Scholes, we infer jump and volatility states by matching a cross-section of market option prices. The same cleaned option universe is then repriced by every candidate, so the comparison is instrument-by-instrument rather than a loose narrative comparison.

Table 7: Classical Merton calibration against the Black-Scholes baseline

Currency	Cal. σ	Cal. λ	Cal. μ_J	Cal. δ_J	BS MAE	Merton MAE
BTC	3.00%	40.05	-2.517%	6.355%	1149.38	230.18
ETH	32.90%	40.05	-3.494%	6.099%	30.96	12.94

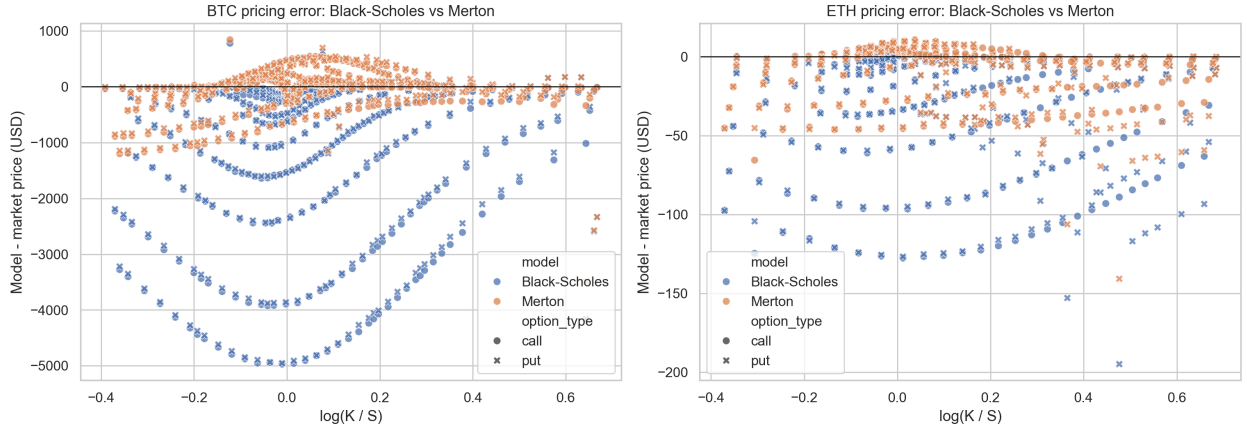


Figure 4: Model pricing errors against Deribit option marks.

Table 8: Candidate-level paired tests versus Black-Scholes absolute error

Currency	Model	Pairs	Mean impr.	Median impr.	Share impr.	t-test p	Wilcoxon p
BTC	Merton	688	919.20	260.68	81.54%	2.18×10^{-67}	2.26×10^{-85}
BTC	SVCJ proxy	688	918.29	235.39	82.27%	2.46×10^{-65}	1.52×10^{-86}
BTC	Dynamic jump	688	881.75	312.51	85.76%	3.69×10^{-75}	1.51×10^{-95}
BTC	MR-ISVM surface	688	781.36	292.08	80.09%	2.16×10^{-62}	1.54×10^{-68}
BTC	GARCH variance	688	522.90	140.25	77.33%	7.84×10^{-71}	1.01×10^{-75}
ETH	MR-ISVM surface	474	19.70	4.54	81.01%	1.27×10^{-40}	1.10×10^{-42}
ETH	Merton	474	18.02	3.25	77.43%	4.03×10^{-44}	1.35×10^{-38}
ETH	Dynamic jump	474	17.42	3.59	77.64%	5.48×10^{-45}	2.12×10^{-39}
ETH	GARCH variance	474	17.08	3.57	85.65%	3.55×10^{-48}	2.97×10^{-53}
ETH	SVCJ proxy	474	0.77	0.00	51.27%	2.02×10^{-1}	9.14×10^{-1}

Table 9: Full candidate pricing comparison on the cleaned option universe

Currency	Model	Options	Bias	MAE	Median AE	RMSE	P90 AE
BTC	Merton	688	-30.80	230.18	128.37	377.22	518.84
BTC	SVCJ proxy	688	7.57	231.09	131.31	373.41	575.66
BTC	Dynamic jump	688	-156.60	267.63	112.46	464.34	787.05
BTC	MR-ISVM surface	688	-113.85	368.03	121.56	752.12	1061.42
BTC	GARCH variance	688	-616.22	626.48	268.89	982.65	1969.18
BTC	Black-Scholes	688	-1139.17	1149.38	498.17	1803.52	3612.63
ETH	MR-ISVM surface	474	-2.27	11.26	4.93	21.23	33.39
ETH	Merton	474	-9.58	12.94	5.28	21.47	40.31
ETH	Dynamic jump	474	-10.47	13.54	5.04	22.51	42.28
ETH	GARCH variance	474	-12.33	13.88	4.54	23.49	42.52
ETH	SVCJ proxy	474	26.31	30.19	8.87	46.48	91.95
ETH	Black-Scholes	474	-30.76	30.96	10.89	49.51	95.11

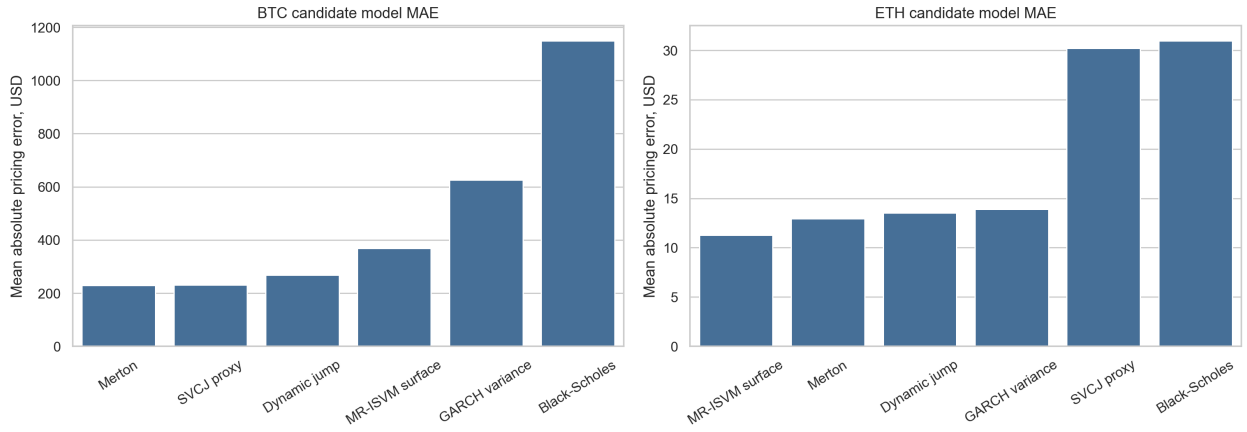


Figure 5: Equal-level candidate comparison using mean absolute pricing error.

Inference. The pricing improvement is not merely visual. Paired t-tests, Wilcoxon signed-rank tests, and sign tests show that most candidates reduce absolute pricing error relative to the Black-Scholes historical-volatility baseline. In this snapshot, Merton reduces BTC MAE from USD 1149.38 to USD 230.18; MR-ISVM reduces ETH MAE from USD 30.96 to USD 11.26.

The paired design is important. Each option is priced once by Black-Scholes and once by every candidate, so the test compares models instrument by instrument rather than comparing unrelated samples. This mirrors the course’s emphasis on using a benchmark model first, then measuring model error against market prices.

Comparative lens. The broader candidate table changes the interpretation. Merton gives the best BTC MAE because the BTC cross section rewards a structural downside-jump premium. SVCJ is almost tied on BTC MAE and has the slightly lower BTC RMSE, meaning it helps on larger pricing misses but is marginally more expensive on average. MR-ISVM gives the best ETH MAE because the live ETH surface is better represented by a stress-regime implied-volatility map than by one jump distribution. GARCH improves materially over Black-Scholes by capturing volatility persistence, but it does not dominate the explicitly jump-aware or surface-aware candidates. The SVCJ proxy is not the ETH marking winner because it prices volatility-jump stress aggressively; that same conservatism becomes valuable in the crash-protection section.

Table 10: Selected calibrated states for the modern candidates

Model	BTC fitted state	ETH fitted state
Merton	$\sigma = 0.0300, \lambda = 40.05, \mu_J = -0.0252, \delta_J = 0.0635$	$\sigma = 0.3290, \lambda = 40.05, \mu_J = -0.0349, \delta_J = 0.0610$
SVCJ proxy	$v_0 = 0.0345, \theta = 0.0009, \xi$ calibrated, multiplier = 0.250	$v_0 = 0.0989, \theta = 0.1082, \xi$ calibrated, multiplier = 0.250
MR-ISVM	active stress, two fitted fallback regimes, 12 coefficients, RMSE = 0.3699	active stress, two regimes, 12 coefficients, RMSE = 0.2783
Dynamic jump	$\sigma = 0.1586, \bar{\lambda} = 31.76, \lambda_t = 31.76, \beta = 2.05$	$\sigma = 0.3602, \bar{\lambda} = 33.01, \lambda_t = 34.51, \beta = 0.25$
GARCH variance	$\alpha + \beta = 0.981$	$\alpha + \beta = 0.959$

8 Interest Rate Sensitivity

We price 30-day at-the-money calls and puts under $r = 1\%$ and $r = 5\%$ to isolate interest-rate sensitivity. The table reports the price change from moving the annual risk-free rate from 1% to 5%, using the same calibrated state for every model.

Table 11: All-candidate interest-rate sensitivity, $V(r = 5\%) - V(r = 1\%)$

Model	BTC call	BTC put	ETH call	ETH put
Black-Scholes	125.32	-126.96	3.35	-3.59
Merton	133.12	-119.15	3.47	-3.48
SVCJ proxy	132.50	-119.78	3.41	-3.53
MR-ISVM surface	123.55	-129.59	3.36	-3.61
Dynamic jump	132.30	-119.97	3.45	-3.50
GARCH variance	124.14	-128.14	3.31	-3.63

Inference. Every model has the expected sign from put-call parity and discounting: higher rates increase calls and reduce puts. For a non-expert reader, the intuition is simple: a higher discount rate lowers the present value of the strike paid in the future, helping calls and hurting puts. Over a 30-day crypto horizon, however, the difference between models is driven much more by volatility, jumps, and surface shape than by the interest-rate input. This is a useful hierarchy of risks: for short-dated BTC/ETH options, the main modeling battle is not the risk-free rate, but the distribution of the underlying.

Comparative lens. The same hierarchy applies to the candidate models. Interest rates are a shared input, while the dominant disagreement across Black-Scholes, Merton, MR-ISVM, SVCJ, dynamic jump intensity, and GARCH comes from the assumed distribution of S_T . This is why we evaluate both average pricing error and standardized crash-protection values.

9 Jump-Parameter Sensitivity

We stress the calibrated Merton model by repricing 30-day, 5% out-of-the-money puts under different jump-intensity and jump-volatility multipliers. This directly measures the value of crash protection.

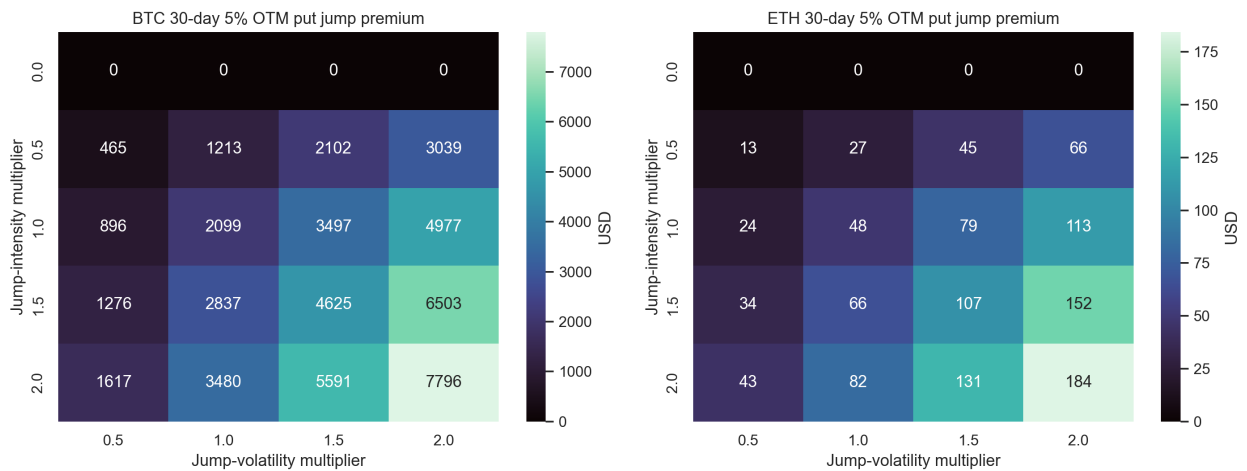


Figure 6: Sensitivity of 30-day 5% OTM put jump premium to jump intensity and jump-size volatility.

Inference. The jump premium rises nonlinearly as either jump intensity or jump-size volatility increases. For BTC, doubling both calibrated jump intensity and jump volatility increases the 30-day 5% OTM put premium by about USD 7796 relative to the low-diffusion baseline implied by the calibrated Merton state. For ETH, the comparable premium is about USD 184. This is a compact risk-management result: crash protection is primarily a jump-risk instrument, not just a high-volatility instrument.

The Risk Management chapter brings this sensitivity analysis into model-risk language. Greeks measure local sensitivities, but a jump parameter shock is a structural stress test. The heatmap

therefore complements Delta and Vega: it asks how much the option value changes when the assumed tail process itself becomes more severe.

Table 12: Standardized 30-day 5% OTM put values across candidates

Currency	Model	Put value	Premium over BS
BTC	Black-Scholes	900.47	0.00
BTC	Merton	2098.74	1198.27
BTC	SVCJ proxy	2145.55	1245.08
BTC	MR-ISVM surface	1963.21	1062.74
BTC	Dynamic jump	1960.79	1060.32
BTC	GARCH variance	1249.04	348.57
ETH	Black-Scholes	58.77	0.00
ETH	Merton	83.19	24.41
ETH	SVCJ proxy	92.48	33.70
ETH	MR-ISVM surface	72.37	13.60
ETH	Dynamic jump	81.98	23.21
ETH	GARCH variance	75.45	16.68

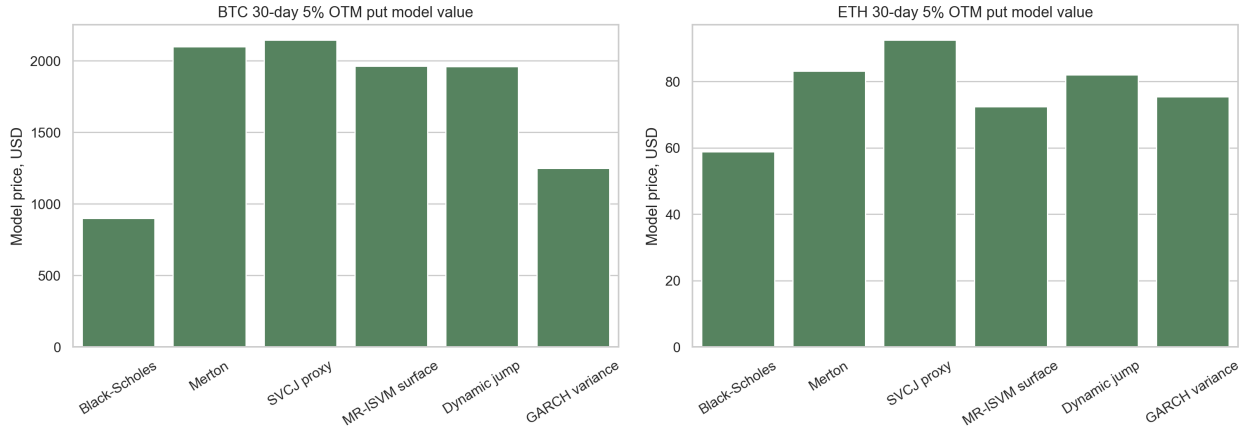


Figure 7: Model-implied crash-protection values for standardized 30-day 5% OTM puts.

Comparative lens. The ranking changes when the objective is crash protection rather than average pricing error. SVCJ produces the largest 30-day downside protection value because it prices the joint event “spot falls and volatility jumps.” Merton and dynamic jump intensity also attach large value to discontinuous downside moves. GARCH is more conservative because volatility clustering alone does not create jump losses. MR-ISVM sits between these cases: it reads the current market smile, but it does not explicitly simulate jump arrival.

10 Greeks and Risk Management

In the course framework, Greeks are sensitivities of option value to state variables and parameters. We compute Black-Scholes Greeks analytically and use finite differences for the candidate models so that the comparison is on the same footing. For the modern candidates, Vega is reported as a *parallel volatility-input Vega*: the value change for a small bump to the model’s effective volatility or implied-volatility input while keeping jump parameters fixed.

Table 13: All-candidate Greeks for 30-day at-the-money puts at $r = 1\%$

Currency	Model	Δ	Γ	Parallel vol Vega	Rho
BTC	Black-Scholes	-0.4801	0.0000657	8789.99	-3231.62
BTC	Merton	-0.4303	0.0000432	630.75	-3018.16
BTC	SVCJ proxy	-0.4318	0.0000425	1685.93	-3033.17
BTC	MR-ISVM surface	-0.4483	0.0085474	8783.18	-3281.74
BTC	Dynamic jump	-0.4357	0.0000461	3560.77	-3041.01
BTC	GARCH variance	-0.4780	0.0000557	8787.47	-3253.27
ETH	Black-Scholes	-0.4723	0.0014892	241.64	-90.85
ETH	Merton	-0.4433	0.0012249	148.54	-87.79
ETH	SVCJ proxy	-0.4462	0.0011319	165.38	-89.18
ETH	MR-ISVM surface	-0.4603	0.4778110	241.56	-91.24
ETH	Dynamic jump	-0.4466	0.0012363	164.15	-88.30
ETH	GARCH variance	-0.4683	0.0012685	241.46	-91.68

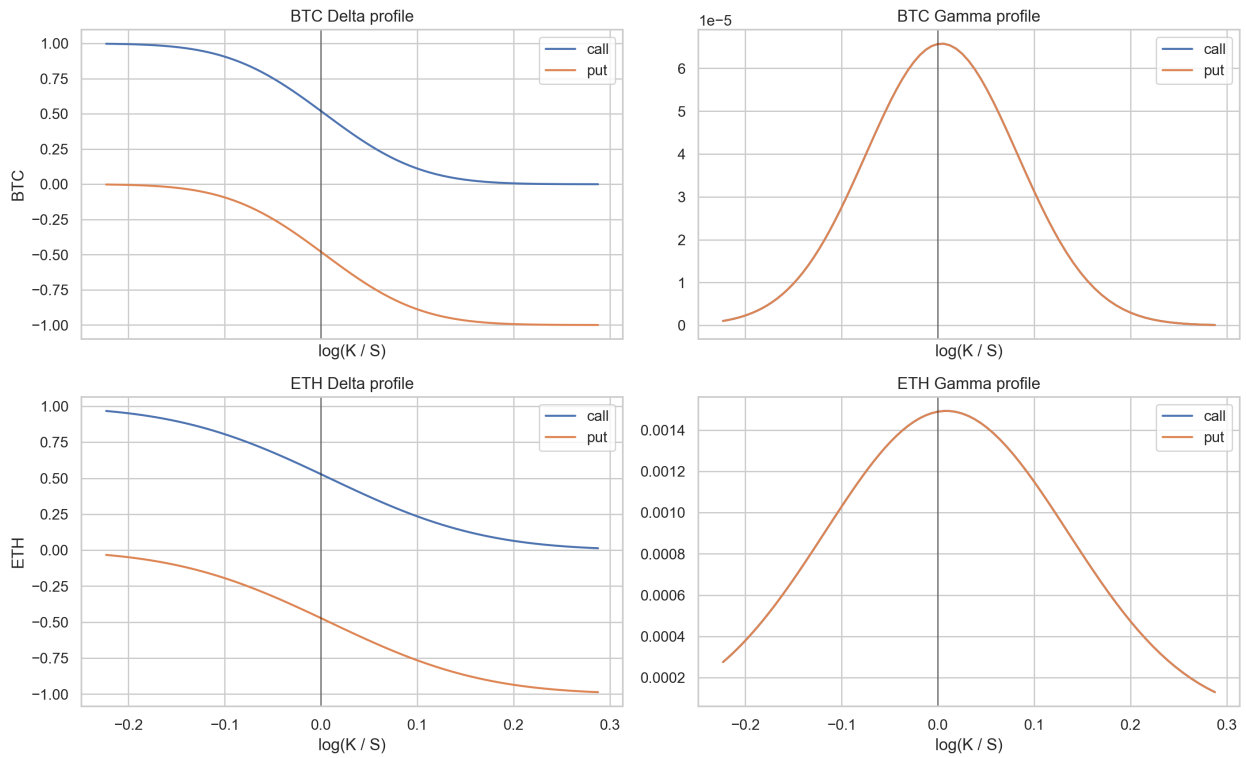


Figure 8: Black-Scholes Delta and Gamma profiles across log-moneyness.

Inference. Jump-aware put deltas are less negative than Black-Scholes deltas because some downside value is carried by jump states rather than only by local diffusion. In plain language, the model says: “not every crash can be hedged by a tiny spot move.” Gamma and Vega are also reallocated. Merton has low parallel-volatility Vega because much of the OTM put value is explained by jump intensity and jump size. MR-ISVM shows very high finite-difference Gamma because its surface delta includes the fitted smile changing as moneyness changes; this is a surface-risk Greek, not merely the textbook Black-Scholes curvature.

The course relation

$$\Theta + rS\Delta + \frac{1}{2}\sigma^2 S^2 \Gamma = r\Pi$$

is a useful interpretation device. Under pure diffusion, Delta and Gamma summarize the first two local spot risks. Under jumps, however, a finite move in S can dominate the Taylor approximation. This is why the Greeks section must be read together with the jump-sensitivity and hedge-simulation

sections.

Comparative lens. Greeks are local diagnostics, while candidate-model differences are partly global distributional assumptions. Black-Scholes and GARCH mainly modify the volatility channel; Merton and dynamic jump intensity modify the jump-arrival channel; SVCJ modifies both variance and jump channels; MR-ISVM embeds the market’s current state into the implied-volatility surface. The practical implication is that a desk should not interpret Delta or Vega without also checking which model generated the distribution around those Greeks.

11 Jump-Hedging Simulation

Continuous delta hedging is an idealization. The Risk Management chapter notes that periodic hedging introduces replication error. We simulate 30-day, 5% OTM put hedging under the same Merton jump paths for every candidate model. Each model supplies its own finite-difference Delta, and the hedge is rebalanced daily. This makes the comparison practical: it asks which model leaves the smaller residual P&L when the underlying path contains jumps.

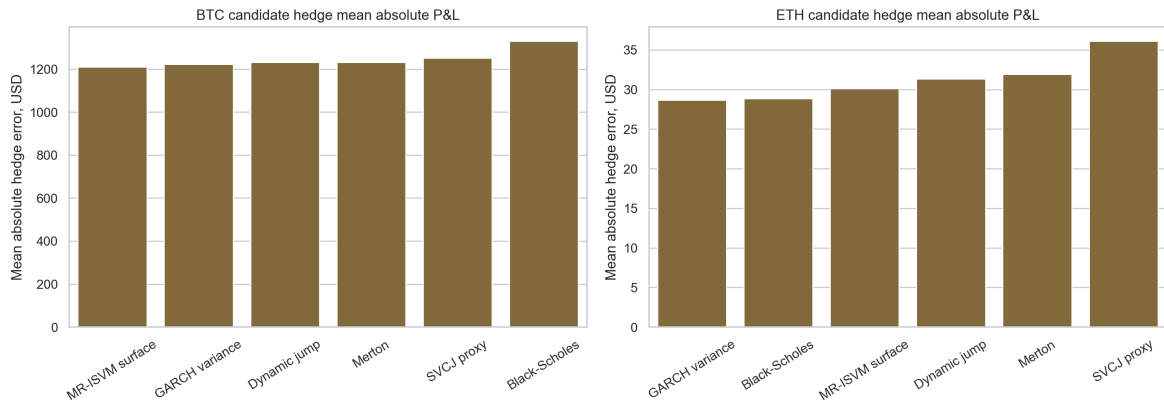


Figure 9: Mean absolute hedge error for all candidates under common Merton jump paths.

Table 14: Candidate-level discrete hedge-error summary

Currency	Model	Mean P&L	Median P&L	Mean P&L	5% P&L
BTC	MR-ISVM surface	-71.09	430.62	1209.79	-3642.82
BTC	GARCH variance	-721.22	-99.04	1221.90	-4115.41
BTC	Dynamic jump	-76.15	532.40	1232.12	-3433.50
BTC	Merton	45.42	597.32	1232.37	-3265.26
BTC	SVCJ proxy	92.57	634.94	1251.67	-3204.44
BTC	Black-Scholes	-1053.16	-439.59	1330.13	-4812.70
ETH	GARCH variance	-0.67	12.53	28.65	-66.09
ETH	Black-Scholes	-17.09	0.77	28.84	-95.27
ETH	MR-ISVM surface	-2.42	14.01	30.11	-70.25
ETH	Dynamic jump	6.09	17.00	31.33	-60.94
ETH	Merton	7.25	17.87	31.92	-60.59
ETH	SVCJ proxy	15.94	25.66	36.12	-51.01

Inference. Hedging gives a different ranking from pricing. **MR-ISVM has the lowest BTC mean absolute hedge error**, while **GARCH has the lowest ETH mean absolute hedge error**. SVCJ has the least negative 5% tail P&L for both assets, but its larger protection premium can raise average hedge error. This is a desk-level lesson: the best marking model, best stress model, and best hedge model need not be the same.

This result connects directly to the replication logic in the binomial and Black-Scholes chapters. Replication is exact only under the assumed dynamics and rebalancing frequency. Once the true path contains discontinuities, a self-financing delta hedge accumulates residual error. The practical question is therefore not whether hedging eliminates risk, but which model leaves the smaller residual under realistic crypto paths.

Comparative lens. All candidates are tested on the same simulated paths, but their economic roles differ. MR-ISVM adapts local deltas to the live surface; Merton and dynamic jump intensity make jump exposure explicit; SVCJ over-insures crash states and improves left-tail hedge P&L; GARCH reacts through persistent conditional variance. Together, the comparison separates three desk questions: which model marks best, which model stresses tail protection most, and which model gives the most stable hedge.

12 Discussion

Black-Scholes is internally coherent under frictionless trading, continuous hedging, constant volatility, and lognormal diffusion. Crypto markets violate these assumptions in several ways:

- returns are heavy-tailed and negatively skewed;
- volatility is regime-dependent;
- investors pay for downside crash protection;
- delta hedging is discrete and liquidity can vanish during stress;
- leverage and liquidation effects can amplify short-horizon moves.

The Merton model is therefore not an ornamental extension. It is a mathematically controlled way to carry the course’s risk-neutral framework into the empirical reality of BTC/ETH options. But the equal-level candidate comparison also shows why Merton should not be the final word. Merton currently gives the cleanest BTC cross-sectional MAE; MR-ISVM gives the cleanest ETH cross-sectional MAE; SVCJ is more explicit about volatility jumps and dominates in standardized crash-premium analysis; dynamic intensity targets jump clustering; and GARCH isolates volatility persistence as a competing explanation. The hedge simulation adds one more layer: a model that is best for marking may not be best for residual hedge P&L.

The sequence of chapters brings the analysis to a layered conclusion: risk-neutral pricing tells us how to value claims once dynamics are specified; binomial trees show how replication works discretely; Black-Scholes gives the diffusion benchmark; implied volatility reveals the benchmark’s market misfit; and risk management exposes the hedging cost of that misfit. The model-comparison layer is our response to that chain of evidence.

13 Conclusion

This project implements a complete option pricing workflow: Black-Scholes prices and Greeks, CRR European and American trees, implied-volatility smiles, market calibration, candidate model comparison, and dynamic hedge simulation.

The innovation is no longer only the Merton jump-diffusion layer. It is the equal-level comparison of six pricing views: diffusion, constant jump intensity, stochastic volatility jumps, regime implied volatility, dynamic jump intensity, and GARCH conditional variance.

The central finding is that crypto option markets price jump risk and volatility-regime risk explicitly. Black-Scholes should be treated as a control model and teaching benchmark. A professional crypto options desk would not select one model for every purpose: it would use the best surface or structural

model for marking, then retain jump and SVCJ-style engines for stress testing, Greeks, and risk management.

References

- [1] Bates, D. S. (1996). Jumps and stochastic volatility: Exchange rate processes implicit in Deutsche Mark options. *The Review of Financial Studies*, 9(1), 69–107.
- [2] Black, F., and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [3] Chen, K. S., and Yang, J. J. (2024). Price dynamics and volatility jumps in bitcoin options. *Financial Innovation*, 10, Article 132.
- [4] Cox, J. C., Ross, S. A., and Rubinstein, M. (1979). Option pricing: A simplified approach. *Journal of Financial Economics*, 7(3), 229–263.
- [5] Duffie, D., Pan, J., and Singleton, K. (2000). Transform analysis and asset pricing for affine jump-diffusions. *Econometrica*, 68(6), 1343–1376.
- [6] Hou, A. J., Wang, W. N., Chen, C. Y.-H., and Härdle, W. K. (2020). Pricing cryptocurrency options. *Journal of Financial Econometrics*, 18(2), 250–279.
- [7] Jiang, W. (2026). IEDA4331 Financial Engineering lecture notes: risk-neutral pricing, binomial trees, Black-Scholes-Merton, and risk management. Hong Kong University of Science and Technology.
- [8] Merton, R. C. (1973). Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1), 141–183.
- [9] Merton, R. C. (1976). Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1–2), 125–144.
- [10] Saef, D., Wang, Y., and Aste, T. (2022). Regime-based implied stochastic volatility model for crypto option pricing. arXiv:2208.12614.
- [11] Venter, P. J., and Maré, E. (2021). Univariate and multivariate GARCH models applied to Bitcoin futures option pricing. *Journal of Risk and Financial Management*, 14(6), Article 261.
- [12] Zulfiqar, N., and Gulzar, S. (2021). Implied volatility estimation of Bitcoin options and the stylized facts of option pricing. *Financial Innovation*, 7, Article 67.

A Implementation Notes

The implementation is modular:

- `src/data_deribit.py`: public Deribit data access and caching.
- `src/preprocessing.py`: quote cleaning, USD conversion, returns, and jump detection.
- `src/black_scholes.py`, `src/binomial.py`, and `src/merton_jump.py`: baseline and jump pricing engines.
- `src/candidate_models.py`: comparable modern candidates.
- `src/calibration.py` and `src/risk.py`: calibration and hedge simulation.
- `scripts/generate_report_assets.py`: reproducible report tables and figures.

B Core Programming Code Listings

This appendix records the key programming modules that implement the report's pricing, calibration, candidate-model, and risk-management analysis. The listings are selected source excerpts rather than every helper function in the repository; each one is labelled so the implementation can be traced directly to the mathematical sections of the report.

```

65     def get_json(
66         self,
67         method: str,
68         params: Mapping[str, Any] | None = None,
69         *,
70         use_cache: bool = True,
71     ) -> dict[str, Any]:
72         """Call a public Deribit method and return the decoded JSON payload."""
73
74         params = {k: v for k, v in (params or {}).items() if v is not None}
75         cache_path = self._cache_path(method, params)
76
77         if use_cache and cache_path and cache_path.exists():
78             age = time.time() - cache_path.stat().st_mtime
79             if age <= self.ttl_seconds:
80                 return json.loads(cache_path.read_text())
81
82         url = f"{self.base_url.rstrip('/')}/{method}"
83         if requests is not None:
84             response = requests.get(url, params=params, timeout=self.timeout)
85             response.raise_for_status()
86             payload = response.json()
87         else: # pragma: no cover
88             query = urllib.parse.urlencode(params)
89             request_url = f"{url}?{query}" if query else url
90             try:
91                 with urllib.request.urlopen(request_url, timeout=self.timeout) as response:
92                     payload = json.loads(response.read().decode("utf-8"))
93             except urllib.error.URLError as exc:
94                 raise DeribitAPIError(f"Deribit request failed: {exc}") from exc
95
96         if "error" in payload:
97             raise DeribitAPIError(str(payload["error"]))
98         if "result" not in payload:
99             raise DeribitAPIError(f"Unexpected Deribit response: {payload}")
100
101         if use_cache and cache_path:
102             cache_path.parent.mkdir(parents=True, exist_ok=True)
103             cache_path.write_text(json.dumps(payload, indent=2, sort_keys=True))
104
105         return payload

```

Listing 1: Data client for Deribit public market-data requests.

```

29     def normalize_option_chain(
30         book: pd.DataFrame,
31         instruments: pd.DataFrame,
32         *,
33         valuation_time: pd.Timestamp | None = None,
34     ) -> pd.DataFrame:
35         """Merge Deribit book summaries with instrument metadata and add features."""
36
37         if book.empty:
38             return pd.DataFrame()
39
40         valuation_time = valuation_time or pd.Timestamp.utcnow()
41         valuation_ms = int(valuation_time.timestamp() * 1000)
42         instrument_cols = [
43             col
44             for col in [
45                 "instrument_name",
46                 "strike",
47                 "option_type",
48                 "expiration_timestamp",
49                 "contract_size",
50                 "settlement_currency",

```

```

51     "quote_currency",
52     ]
53     if col in instruments.columns
54     ]
55     df = book.copy()
56     if instrument_cols:
57         df = df.merge(instruments[instrument_cols], on="instrument_name", how="left")
58
59     parsed = df["instrument_name"].map(_parse_instrument_name)
60     parsed_strike = parsed.map(lambda item: item[0])
61     parsed_type = parsed.map(lambda item: item[1])
62     df["strike"] = pd.to_numeric(df.get("strike", parsed_strike), errors="coerce").fillna(parsed_strike)
63     df["option_type"] = df.get("option_type", parsed_type).fillna(parsed_type).str.lower()
64     if "expiration_timestamp" not in df:
65         df["expiration_timestamp"] = np.nan
66
67     numeric_cols = [
68         "bid_price",
69         "ask_price",
70         "mid_price",
71         "mark_price",
72         "underlying_price",
73         "interest_rate",
74         "open_interest",
75         "volume",
76         "mark_iv",
77         "expiration_timestamp",
78     ]
79     for col in numeric_cols:
80         if col in df:
81             df[col] = pd.to_numeric(df[col], errors="coerce")
82
83     df["valuation_timestamp"] = valuation_ms
84     df["valuation_datetime"] = pd.to_datetime(valuation_ms, unit="ms", utc=True)
85     df["expiration_datetime"] = pd.to_datetime(df["expiration_timestamp"], unit="ms", utc=True)
86     df["time_to_maturity"] = (df["expiration_timestamp"] - valuation_ms) / MS_PER_YEAR
87     df["rate"] = pd.Series(_normalise_percent(df.get("interest_rate", 0.0)), index=df.index).fillna(0.0)
88     df["mark_iv_decimal"] = pd.Series(_normalise_percent(df.get("mark_iv", np.nan)), index=df.index)
89
90     for col in ["bid_price", "ask_price", "mid_price", "mark_price"]:
91         if col in df:
92             df[f"{col}_usd"] = df[col] * df["underlying_price"]
93
94     mid = df.get("mid_price")
95     if mid is None:
96         mid = (df["bid_price"] + df["ask_price"]) / 2.0
97     derived_mid = (df["bid_price"] + df["ask_price"]) / 2.0
98     df["market_price_coin"] = np.where(mid > 0.0, mid, df["mark_price"])
99     df["market_price_coin"] = np.where(df["market_price_coin"] > 0.0, df["market_price_coin"], derived_mid)
100    df["market_price_usd"] = df["market_price_coin"] * df["underlying_price"]
101    df["spread_coin"] = df["ask_price"] - df["bid_price"]
102    df["spread_usd"] = df["spread_coin"] * df["underlying_price"]
103    df["moneyness"] = df["underlying_price"] / df["strike"]
104    df["log_moneyness"] = np.log(df["strike"] / df["underlying_price"])
105    df["liquidity_score"] = df[["open_interest", "volume"]].fillna(0.0).sum(axis=1)
106    return df.sort_values(["expiration_timestamp", "strike", "option_type"]).reset_index(drop=True)

```

Listing 2: Option-chain normalization: metadata merge, USD conversion, maturity, moneyness, and IV features.

```

135 def compute_log_returns(index_history: pd.DataFrame) -> pd.DataFrame:
136     """Compute timestamped log returns from a Deribit index history frame."""
137
138     df = index_history.sort_values("timestamp").copy()
139     df["log_price"] = np.log(df["price"])
140     df["log_return"] = df["log_price"].diff()
141     return df.dropna(subset=["log_return"]).reset_index(drop=True)
142
143
144 def detect_jumps(
145     returns: pd.DataFrame,
146     *,
147     z_threshold: float = 3.0,
148     return_col: str = "log_return",
149 ) -> pd.DataFrame:

```

```

150     """Flag large absolute return shocks using a robust z-score."""
151
152     df = returns.copy()
153     median = df[return_col].median()
154     mad = (df[return_col] - median).abs().median()
155     robust_sigma = 1.4826 * mad if mad > 0.0 else df[return_col].std(ddof=1)
156     df["jump_z"] = (df[return_col] - median) / max(robust_sigma, 1e-12)
157     df["is_jump"] = df["jump_z"].abs() >= z_threshold
158     return df

```

Listing 3: Historical return construction and robust jump detection.

```

53 def d1_d2(S: Any, K: Any, T: Any, r: Any, sigma: Any, q: Any = 0.0):
54     """Return Black-Scholes ``d1`` and ``d2`` arrays."""
55
56     S, K, T, r, sigma, q = _broadcast_inputs(S, K, T, r, sigma, q)
57     sqrt_T = np.sqrt(np.maximum(T, 1e-16))
58     vol_sqrt_T = np.maximum(sigma * sqrt_T, 1e-16)
59     d1 = (np.log(S / K) + (r - q + 0.5 * sigma * sigma) * T) / vol_sqrt_T
60     d2 = d1 - vol_sqrt_T
61     return d1, d2
62
63
64 def bs_price(
65     S: Any,
66     K: Any,
67     T: Any,
68     r: Any,
69     sigma: Any,
70     option_type: Any = "call",
71     q: Any = 0.0,
72 ) -> float | np.ndarray:
73     """Black-Scholes price with optional continuous carry/dividend yield ``q``."""
74
75     S, K, T, r, sigma, q = _broadcast_inputs(S, K, T, r, sigma, q)
76     calls = np.broadcast_to(_is_call(option_type), S.shape)
77     call_intrinsic = np.maximum(S - K, 0.0)
78     put_intrinsic = np.maximum(K - S, 0.0)
79     intrinsic = np.where(calls, call_intrinsic, put_intrinsic)
80
81     valid = (S > 0.0) & (K > 0.0) & (T > 0.0) & (sigma > 0.0)
82     out = intrinsic.astype(float)
83     if np.any(valid):
84         d1, d2 = d1_d2(S[valid], K[valid], T[valid], r[valid], sigma[valid], q[valid])
85         discounted_spot = S[valid] * np.exp(-q[valid] * T[valid])
86         discounted_strike = K[valid] * np.exp(-r[valid] * T[valid])
87         call = discounted_spot * _normal_cdf(d1) - discounted_strike * _normal_cdf(d2)
88         put = discounted_strike * _normal_cdf(-d2) - discounted_spot * _normal_cdf(-d1)
89         out[valid] = np.where(calls[valid], call, put)
90     return _maybe_scalar(out)
91
92
93 def bs_greeks(
94     S: Any,
95     K: Any,
96     T: Any,
97     r: Any,
98     sigma: Any,
99     option_type: Any = "call",
100     q: Any = 0.0,
101 ) -> dict[str, float | np.ndarray]:
102     """Return Delta, Gamma, Vega, Theta, and Rho.
103
104     Vega and Rho are reported per unit change in volatility/rate, not per
105     one-percentage-point change.
106     """
107
108     S, K, T, r, sigma, q = _broadcast_inputs(S, K, T, r, sigma, q)
109     calls = np.broadcast_to(_is_call(option_type), S.shape)
110     valid = (S > 0.0) & (K > 0.0) & (T > 0.0) & (sigma > 0.0)
111
112     delta = np.where(calls, (S > K).astype(float), -(S < K).astype(float))
113     gamma = np.full(S.shape, np.nan)
114     vega = np.full(S.shape, np.nan)
115     theta = np.full(S.shape, np.nan)
116     rho = np.full(S.shape, np.nan)

```

```

117
118 if np.any(valid):
119     d1, d2 = d1_d2(S[valid], K[valid], T[valid], r[valid], sigma[valid], q[valid])
120     sqrt_T = np.sqrt(T[valid])
121     exp_q = np.exp(-q[valid] * T[valid])
122     exp_r = np.exp(-r[valid] * T[valid])
123     pdf_d1 = _normal_pdf(d1)
124     nd1 = _normal_cdf(d1)
125     nd2 = _normal_cdf(d2)
126     nmd1 = _normal_cdf(-d1)
127     nmd2 = _normal_cdf(-d2)
128
129     delta_call = exp_q * nd1
130     delta_put = exp_q * (nd1 - 1.0)
131     gamma_v = exp_q * pdf_d1 / (S[valid] * sigma[valid] * sqrt_T)
132     vega_v = S[valid] * exp_q * pdf_d1 * sqrt_T
133
134     theta_common = -S[valid] * exp_q * pdf_d1 * sigma[valid] / (2.0 * sqrt_T)
135     theta_call = (
136         theta_common
137         - r[valid] * K[valid] * exp_r * nd2
138         + q[valid] * S[valid] * exp_q * nd1
139     )
140     theta_put = (
141         theta_common
142         + r[valid] * K[valid] * exp_r * nmd2
143         - q[valid] * S[valid] * exp_q * nmd1
144     )
145     rho_call = K[valid] * T[valid] * exp_r * nd2
146     rho_put = -K[valid] * T[valid] * exp_r * nmd2
147
148     delta[valid] = np.where(calls[valid], delta_call, delta_put)
149     gamma[valid] = gamma_v
150     vega[valid] = vega_v
151     theta[valid] = np.where(calls[valid], theta_call, theta_put)
152     rho[valid] = np.where(calls[valid], rho_call, rho_put)
153
154 return {
155     "delta": _maybe_scalar(delta),
156     "gamma": _maybe_scalar(gamma),
157     "vega": _maybe_scalar(vega),
158     "theta": _maybe_scalar(theta),
159     "rho": _maybe_scalar(rho),
160 }

```

Listing 4: Black-Scholes pricing and analytic Greeks.

```

163 def implied_volatility(
164     market_price: Any,
165     S: Any,
166     K: Any,
167     T: Any,
168     r: Any,
169     option_type: Any = "call",
170     q: Any = 0.0,
171     *,
172     tol: float = 1e-8,
173     max_iter: int = 120,
174     max_vol: float = 10.0,
175 ) -> float | np.ndarray:
176     """Recover Black-Scholes implied volatility using robust bisection."""
177
178     price, S, K, T, r, _, q = np.broadcast_arrays(
179         np.asarray(market_price, dtype=float),
180         np.asarray(S, dtype=float),
181         np.asarray(K, dtype=float),
182         np.asarray(T, dtype=float),
183         np.asarray(r, dtype=float),
184         np.asarray(0.0, dtype=float),
185         np.asarray(q, dtype=float),
186     )
187     calls = np.broadcast_to(_is_call(option_type), price.shape)
188     intrinsic = np.where(calls, np.maximum(S - K, 0.0), np.maximum(K - S, 0.0))
189     valid = (price >= intrinsic - 1e-10) & (S > 0.0) & (K > 0.0) & (T > 0.0)
190     out = np.full(price.shape, np.nan)
191     if not np.any(valid):

```

```

192     return _maybe_scalar(out)
193
194     low = np.full(price.shape, 1e-8)
195     high = np.full(price.shape, 1.0)
196     for _ in range(12):
197         model_high = np.asarray(bs_price(S, K, T, r, high, option_type, q))
198         needs_higher = (model_high < price) & valid & (high < max_vol)
199         if not np.any(needs_higher):
200             break
201         high = np.where(needs_higher, np.minimum(high * 2.0, max_vol), high)
202
203     for _ in range(max_iter):
204         mid = 0.5 * (low + high)
205         model = np.asarray(bs_price(S, K, T, r, mid, option_type, q))
206         low = np.where((model < price) & valid, mid, low)
207         high = np.where((model >= price) & valid, mid, high)
208         if np.nanmax(high[valid] - low[valid]) < tol:
209             break
210
211     out[valid] = 0.5 * (low[valid] + high[valid])
212     return _maybe_scalar(out)

```

Listing 5: Black-Scholes implied-volatility inversion by robust bisection.

```

17 def crr_price(
18     S: float,
19     K: float,
20     T: float,
21     r: float,
22     sigma: float,
23     option_type: str = "call",
24     *,
25     steps: int = 250,
26     style: OptionStyle = "european",
27     q: float = 0.0,
28 ) -> float:
29     """Price a vanilla option with a CRR binomial tree."""
30
31     if T <= 0.0:
32         return max(S - K, 0.0) if option_type.lower().startswith("c") else max(K - S, 0.0)
33     if steps < 1:
34         raise ValueError("steps must be positive")
35     if S <= 0.0 or K <= 0.0 or sigma <= 0.0:
36         raise ValueError("S, K, and sigma must be positive")
37
38     dt = T / steps
39     u = math.exp(sigma * math.sqrt(dt))
40     d = 1.0 / u
41     growth = math.exp((r - q) * dt)
42     p = (growth - d) / (u - d)
43     if p < 0.0 or p > 1.0:
44         raise ValueError("risk-neutral probability is outside [0, 1]")
45
46     j = np.arange(steps + 1)
47     terminal_spots = S * (u ** j) * (d ** (steps - j))
48     is_call = option_type.lower().startswith("c")
49     values = np.maximum(terminal_spots - K, 0.0) if is_call else np.maximum(K - terminal_spots, 0.0)
50     discount = math.exp(-r * dt)
51
52     for level in range(steps - 1, -1, -1):
53         values = discount * (p * values[1 : level + 2] + (1.0 - p) * values[: level + 1])
54         if style == "american":
55             spots = S * (u ** np.arange(level + 1)) * (d ** (level - np.arange(level + 1)))
56             exercise = np.maximum(spots - K, 0.0) if is_call else np.maximum(K - spots, 0.0)
57             values = np.maximum(values, exercise)
58     return float(values[0])
59
60
61 def convergence_table(
62     S: float,
63     K: float,
64     T: float,
65     r: float,
66     sigma: float,
67     option_type: str = "call",
68     *,

```



```

69     steps_grid: tuple[int, ...] = (25, 50, 100, 250, 500, 1000),
70     q: float = 0.0,
71 ) -> pd.DataFrame:
72     """Return CRR convergence diagnostics against Black-Scholes."""
73
74     benchmark = float(bs_price(S, K, T, r, sigma, option_type, q))
75     rows = []
76     for steps in steps_grid:
77         price = crr_price(S, K, T, r, sigma, option_type, steps=steps, q=q)
78         rows.append(
79             {
80                 "steps": steps,
81                 "crr_price": price,
82                 "black_scholes_price": benchmark,
83                 "absolute_error": abs(price - benchmark),
84                 "relative_error": abs(price - benchmark) / max(benchmark, 1e-12),
85             }
86         )
87     return pd.DataFrame(rows)

```

Listing 6: CRR binomial tree engine with European/American exercise support and convergence diagnostics.

```

14 @dataclass(frozen=True)
15 class MertonParams:
16     """Risk-neutral Merton jump-diffusion parameters."""
17
18     sigma: float
19     jump_intensity: float
20     jump_mean: float
21     jump_vol: float
22
23     @property
24     def kappa(self) -> float:
25         return math.exp(self.jump_mean + 0.5 * self.jump_vol * self.jump_vol) - 1.0
26
27
28 def _maybe_scalar(values: np.ndarray) -> float | np.ndarray:
29     return float(values) if values.ndim == 0 else values
30
31
32 def _poisson_weights(lam_t: np.ndarray, max_jumps: int | None, tol: float) -> list[np.ndarray]:
33     if max_jumps is None:
34         lam_max = float(np.nanmax(lam_t)) if lam_t.size else 0.0
35         max_jumps = max(20, int(math.ceil(lam_max + 8.0 * math.sqrt(lam_max + 1.0))))
36     weights: list[np.ndarray] = []
37     weight = np.exp(-lam_t)
38     cumulative = weight.copy()
39     weights.append(weight)
40     for n in range(1, max_jumps + 1):
41         weight = weight * lam_t / n
42         cumulative = cumulative + weight
43         weights.append(weight)
44         if np.nanmax(1.0 - cumulative) < tol:
45             break
46     return weights
47
48
49 def merton_price(
50     S: Any,
51     K: Any,
52     T: Any,
53     r: Any,
54     option_type: Any = "call",
55     *,
56     sigma: float,
57     jump_intensity: float,
58     jump_mean: float,
59     jump_vol: float,
60     max_jumps: int | None = None,
61     poisson_tol: float = 1e-10,
62 ) -> float | np.ndarray:
63     """Price a European vanilla option under Merton jump diffusion.
64
65     The implementation conditions on the number of jumps. Conditional on ``n``

```

```

66 jumps, log-price remains Gaussian with adjusted drift and variance, which
67 can be priced as a generalized Black-Scholes contract with continuous carry.
68 """
69
70 S, K, T, r, sigma, jump_intensity, jump_mean, jump_vol = np.broadcast_arrays(
71     np.asarray(S, dtype=float),
72     np.asarray(K, dtype=float),
73     np.asarray(T, dtype=float),
74     np.asarray(r, dtype=float),
75     np.asarray(sigma, dtype=float),
76     np.asarray(jump_intensity, dtype=float),
77     np.asarray(jump_mean, dtype=float),
78     np.asarray(jump_vol, dtype=float),
79 )
80 if np.all(jump_intensity <= 1e-14):
81     return bs_price(S, K, T, r, sigma, option_type)
82 if np.any(sigma <= 0.0) or np.any(jump_intensity < 0.0) or np.any(jump_vol < 0.0):
83     raise ValueError("sigma and jump_vol must be positive; jump_intensity must be non-negative")
84
85 lam_t = jump_intensity * np.maximum(T, 0.0)
86 kappa = np.exp(jump_mean + 0.5 * jump_vol * jump_vol) - 1.0
87 weights = _poisson_weights(lam_t, max_jumps, poisson_tol)
88 price = np.zeros_like(S, dtype=float)
89
90 for n, weight in enumerate(weights):
91     total_var = sigma * sigma + (n * jump_vol * jump_vol) / np.maximum(T, 1e-16)
92     sigma_n = np.sqrt(np.maximum(total_var, 1e-16))
93     drift_n = r - jump_intensity * kappa + (n * jump_mean + 0.5 * n * jump_vol * jump_vol) / np.maximum(T,
94         1e-16)
95     q_n = r - drift_n
96     price = price + weight * np.asarray(bs_price(S, K, T, r, sigma_n, option_type, q=q_n))
97 return _maybe_scalar(price)

```

Listing 7: Merton jump-diffusion parameters and Poisson-mixture pricing engine.

```

99 def merton_greeks(
100     S: float,
101     K: float,
102     T: float,
103     r: float,
104     option_type: str = "call",
105     *,
106     sigma: float,
107     jump_intensity: float,
108     jump_mean: float,
109     jump_vol: float,
110 ) -> dict[str, float]:
111     """Finite-difference Greeks for the Merton model."""
112
113     if jump_intensity <= 1e-14:
114         return {key: float(value) for key, value in bs_greeks(S, K, T, r, sigma, option_type).items()}
115
116     def price(s=S, k=K, t=T, rate=r, vol=sigma):
117         return float(
118             merton_price(
119                 s,
120                 k,
121                 max(t, 1e-8),
122                 rate,
123                 option_type,
124                 sigma=max(vol, 1e-8),
125                 jump_intensity=jump_intensity,
126                 jump_mean=jump_mean,
127                 jump_vol=jump_vol,
128             )
129         )
130
131     h_s = max(1e-3, 1e-4 * S)
132     h_v = max(1e-5, 1e-4 * sigma)
133     h_r = 1e-4
134     h_t = min(max(1e-5, 1e-4 * T), max(0.5 * T, 1e-5))
135
136     base = price()
137     up_s = price(s=S + h_s)
138     dn_s = price(s=max(S - h_s, 1e-8))
139     delta = (up_s - dn_s) / (2.0 * h_s)

```

```

140 gamma = (up_s - 2.0 * base + dn_s) / (h_s * h_s)
141 vega = (price(vol=sigma + h_v) - price(vol=max(sigma - h_v, 1e-8))) / (2.0 * h_v)
142 rho = (price(rate=r + h_r) - price(rate=r - h_r)) / (2.0 * h_r)
143 theta = (price(t=max(T - h_t, 1e-8)) - price(t=T + h_t)) / (2.0 * h_t)
144 return {"delta": delta, "gamma": gamma, "vega": vega, "theta": theta, "rho": rho}
145
146
147 def simulate_merton_paths(
148     S0: float,
149     T: float,
150     *,
151     r: float,
152     sigma: float,
153     jump_intensity: float,
154     jump_mean: float,
155     jump_vol: float,
156     steps: int = 365,
157     paths: int = 1000,
158     seed: int | None = 4331,
159 ) -> np.ndarray:
160     """Simulate Merton jump-diffusion paths under the risk-neutral measure."""
161
162     rng = np.random.default_rng(seed)
163     dt = T / steps
164     kappa = math.exp(jump_mean + 0.5 * jump_vol * jump_vol) - 1.0
165     out = np.empty((paths, steps + 1), dtype=float)
166     out[:, 0] = S0
167     drift = (r - jump_intensity * kappa - 0.5 * sigma * sigma) * dt
168     diffusion_scale = sigma * math.sqrt(dt)
169     for i in range(1, steps + 1):
170         z = rng.standard_normal(paths)
171         n_jumps = rng.poisson(jump_intensity * dt, paths)
172         jump_log = rng.normal(jump_mean * n_jumps, jump_vol * np.sqrt(n_jumps))
173         out[:, i] = out[:, i - 1] * np.exp(drift + diffusion_scale * z + jump_log)
174     return out

```

Listing 8: Merton finite-difference Greeks and risk-neutral path simulation.

```

41 def estimate_historical_merton_params(
42     index_history: pd.DataFrame,
43     *,
44     z_threshold: float = 3.0,
45 ) -> tuple[MertonParams, pd.DataFrame]:
46     """Estimate initial Merton parameters from historical index returns."""
47
48     returns = detect_jumps(compute_log_returns(index_history), z_threshold=z_threshold)
49     ppy = _periods_per_year(returns["timestamp"])
50     years = max(len(returns) / ppy, 1e-12)
51     non_jump_returns = returns.loc[~returns["is_jump"], "log_return"]
52     jump_returns = returns.loc[returns["is_jump"], "log_return"]
53
54     if len(non_jump_returns) < 3:
55         non_jump_returns = returns["log_return"]
56     sigma = float(non_jump_returns.std(ddof=1) * np.sqrt(ppy))
57     sigma = float(np.clip(sigma, 0.03, 3.0))
58
59     if len(jump_returns) == 0:
60         jump_intensity = 0.25
61         jump_mean = 0.0
62         jump_vol = max(float(returns["log_return"].std(ddof=1) * 2.0), 0.01)
63     else:
64         jump_intensity = float(len(jump_returns) / years)
65         jump_mean = float(jump_returns.mean())
66         jump_vol = float(jump_returns.std(ddof=1)) if len(jump_returns) > 1 else abs(jump_mean)
67         jump_vol = max(jump_vol, 0.01)
68
69     params = MertonParams(
70         sigma=sigma,
71         jump_intensity=float(np.clip(jump_intensity, 1e-4, 250.0)),
72         jump_mean=float(np.clip(jump_mean, -1.0, 1.0)),
73         jump_vol=float(np.clip(jump_vol, 1e-4, 2.0)),
74     )
75     returns.attrs["periods_per_year"] = ppy
76     return params, returns

```

Listing 9: Historical initialization of Merton parameters from index returns.

```

101 def calibrate_merton_to_options(
102     options: pd.DataFrame,
103     initial: MertonParams,
104     *,
105     max_options: int = 250,
106     rate_override: float | None = None,
107 ) -> CalibrationResult:
108     """Calibrate Merton parameters by minimizing weighted option price errors."""
109
110     if minimize is None: # pragma: no cover
111         raise RuntimeError("scipy is required for calibration. Install requirements.txt first.")
112
113     sample = calibration_sample(options, max_options=max_options)
114     if sample.empty:
115         return CalibrationResult(initial, np.nan, np.nan, 0, False, "No usable options")
116
117     S = sample["underlying_price"].to_numpy(float)
118     K = sample["strike"].to_numpy(float)
119     T = sample["time_to_maturity"].to_numpy(float)
120     market = sample["market_price_usd"].to_numpy(float)
121     r = np.full(len(sample), rate_override, dtype=float) if rate_override is not None else sample["rate"].
122         fillna(0.0).to_numpy(float)
123     option_type = sample["option_type"].to_numpy(str)
124     spread = sample.get("spread_usd", pd.Series(np.nan, index=sample.index)).fillna(0.0).to_numpy(float)
125     weights = 1.0 / np.maximum(spread + 0.02 * market, 1.0)
126     weights = weights / np.nanmean(weights)
127
128     bounds = [(0.03, 3.0), (1e-4, 250.0), (-1.0, 1.0), (1e-4, 2.0)]
129     x0 = np.array(
130         [
131             np.clip(initial.sigma, *bounds[0]),
132             np.clip(initial.jump_intensity, *bounds[1]),
133             np.clip(initial.jump_mean, *bounds[2]),
134             np.clip(initial.jump_vol, *bounds[3]),
135         ],
136         dtype=float,
137     )
138
139     def objective(x: np.ndarray) -> float:
140         sigma, jump_intensity, jump_mean, jump_vol = x
141         model = np.asarray(
142             merton_price(
143                 S,
144                 K,
145                 T,
146                 r,
147                 option_type,
148                 sigma=float(sigma),
149                 jump_intensity=float(jump_intensity),
150                 jump_mean=float(jump_mean),
151                 jump_vol=float(jump_vol),
152             )
153         )
154         errors = (model - market) / np.maximum(market, 1.0)
155         return float(np.nanmean(weights * errors * errors))
156
157     result = minimize(objective, x0, method="L-BFGS-B", bounds=bounds, options={"maxiter": 300})
158     params = MertonParams(
159         sigma=float(result.x[0]),
160         jump_intensity=float(result.x[1]),
161         jump_mean=float(result.x[2]),
162         jump_vol=float(result.x[3]),
163     )
164     model = np.asarray(
165         merton_price(
166             S,
167             K,
168             T,
169             r,
170             option_type,
171             sigma=params.sigma,
172             jump_intensity=params.jump_intensity,
173             jump_mean=params.jump_mean,
174             jump_vol=params.jump_vol,
175         )

```

```

175 )
176 errors = model - market
177 return CalibrationResult(
178     params=params,
179     rmse=float(np.sqrt(np.nanmean(errors * errors))),
180     mae=float(np.nanmean(np.abs(errors))),
181     n_options=len(sample),
182     success=bool(result.success),
183     message=str(result.message),
184 )

```

Listing 10: Market calibration of Merton parameters to Deribit option prices.

```

83 def _surface_design(options: pd.DataFrame) -> tuple[np.ndarray, tuple[str, ...]]:
84     x = options.get("log_moneyness")
85     if x is None:
86         x = np.log(options["strike"].to_numpy(float) / options["underlying_price"].to_numpy(float))
87     else:
88         x = x.to_numpy(float)
89     t = np.maximum(options["time_to_maturity"].to_numpy(float), 1e-8)
90     put = options["option_type"].astype(str).str.lower().str.startswith("p").to_numpy(float)
91     short = (t <= 30.0 / 365.25).astype(float)
92     sqrt_t = np.sqrt(t)
93     # Three extra terms added for MR-ISVM:
94     #   log_moneyness_x_sqrt_T : SVI-style cross term - controls how skew
95     #                           steepness varies with maturity (key for term
96     #                           structure of the crypto smile).
97     #   put_x_sqrt_T           : puts and calls have different term structures;
98     #                           this lets the surface tilt the IV term slope
99     #                           separately for each side.
100    #   log_moneyness_cube      : cubic asymmetry - crypto OTM puts are more
101    #                           expensive than the quadratic term alone can
102    #                           capture, driving systematic under-pricing.
103    features = (
104        "intercept",
105        "log_moneyness",
106        "abs_log_moneyness",
107        "log_moneyness_sq",
108        "sqrt_T",
109        "T",
110        "put",
111        "put_x_log_moneyness",
112        "short_maturity",
113        "log_moneyness_x_sqrt_T",
114        "put_x_sqrt_T",
115        "log_moneyness_cube",
116    )
117    design = np.column_stack(
118        [
119            np.ones(len(options)),
120            x,
121            np.abs(x),
122            x * x,
123            sqrt_t,
124            t,
125            put,
126            put * x,
127            short,
128            x * sqrt_t,          # SVI cross term
129            put * sqrt_t,       # put-side term slope
130            x * x * x,          # cubic skew asymmetry
131        ]
132    )
133    return design, features
134
135
136 def fit_implied_surface(
137     options: pd.DataFrame,
138     *,
139     max_options: int = 80,
140     ridge: float = 1e-3,
141 ) -> ImpliedSurfaceParams:
142     """
143     Fit a reduced MR-ISVM implied-volatility surface on a calibration sample.
144
145     Changes from the original

```

```

146 -----
147 Fix 1 - price-space objective from the start, no IV warm-start
148 The original code fitted an OLS solution in IV space and used it as a
149 warm-start for a price-space refinement. The OLS warm-start lands in
150 a local minimum of the IV loss, which is a different landscape from the
151 price loss. L-BFGS-B then does not move far enough to escape it.
152
153 The new code initialises from a physically meaningful flat smile
154 (intercept = median ATM IV, all other coefficients = 0) and runs
155 L-BFGS-B directly on the price-space weighted RMSE objective with
156 2 000 iterations. This avoids the IV-space local minimum entirely.
157
158 Fix 2 - vega-weighted objective instead of inverse-price weights
159 The original weights = 1 / max(market_price, 0.5) give equal relative
160 attention across strikes. But USD RMSE is dominated by high-vega
161 near-ATM options. Fitting with inverse-price weights over-emphasises
162 cheap deep-OTM options whose price errors are tiny in USD terms,
163 causing the surface to under-fit ATM where the error budget matters.
164
165 The new code weights each option by its approximate Black-Scholes vega
166 (dC/dsigma approx S * phi(d1) * sqrt(T)). This makes the calibration objective
167 proportional to how much a given IV error contributes to USD price
168 error, aligning the calibration loss with the evaluation metric.
169 """
170 sample = calibration_sample(options, max_options=max_options)
171 sample = sample.dropna(subset=["mark_iv_decimal"]).copy()
172 sample = sample[sample["mark_iv_decimal"].between(0.01, 4.0)]
173 if sample.empty:
174     raise ValueError("No usable options for implied-surface calibration")
175
176 X, feature_names = _surface_design(sample)
177 X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)
178
179 S_arr = sample["underlying_price"].to_numpy(float)
180 K_arr = sample["strike"].to_numpy(float)
181 T_arr = np.maximum(sample["time_to_maturity"].to_numpy(float), 1e-8)
182 r_arr = sample["rate"].fillna(0.0).to_numpy(float)
183 opt_arr = sample["option_type"].to_numpy(object)
184 T_days = T_arr * 365.25
185
186 market_price = sample["market_price_usd"].to_numpy(float)
187
188 sigma_floor = 0.20
189 sigma_cap = 3.0
190
191 # -----
192 # Fix 2: vega-based weights
193 # -----
194 # Approximate ATM sigma per option from mark_iv_decimal; fall back to
195 # the sample median when the column is missing or NaN.
196 sigma_atm = sample["mark_iv_decimal"].fillna(
197     sample["mark_iv_decimal"].median()
198 ).to_numpy(float)
199 sigma_atm = np.clip(sigma_atm, 0.05, 3.0)
200
201 # Black-Scholes vega approx S * phi(d1) * sqrt(T) (independent of call/put)
202 d1 = (
203     np.log(S_arr / K_arr)
204     + (r_arr + 0.5 * sigma_atm ** 2) * T_arr
205 ) / (sigma_atm * np.sqrt(T_arr))
206 vega_approx = S_arr * np.exp(-0.5 * d1 ** 2) / np.sqrt(2.0 * np.pi) * np.sqrt(T_arr)
207 vega_approx = np.maximum(vega_approx, 1e-4)
208
209 # Normalise so the average weight = 1 (keeps objective scale stable)
210 weights = vega_approx / vega_approx.mean()
211 weights = np.clip(weights, 0.05, 20.0)
212
213 # -----
214 # Helper: map coefficient vector -> clipped IV -> BS price array
215 # -----
216 def _price_from_coefs(c: np.ndarray) -> np.ndarray:
217     sigma_iv = np.einsum("ij,j->i", X, c)
218     adaptive_floor = np.maximum(
219         sigma_floor,
220         0.15 * np.sqrt(np.maximum(30.0 / T_days, 1.0)),
221     )

```

```

222     sigma_iv = np.clip(sigma_iv, adaptive_floor, sigma_cap)
223     return np.asarray(bs_price(S_arr, K_arr, T_arr, r_arr, sigma_iv, opt_arr), dtype=float)
224
225     # -----
226     # Price-space objective (vega-weighted relative RMSE)
227     # -----
228     def objective(c: np.ndarray) -> float:
229         model_px = _price_from_coefs(c)
230         rel_err = (model_px - market_price) / np.maximum(market_price, 0.5)
231         return float(np.sqrt(np.average(rel_err ** 2, weights=weights)))
232
233     # -----
234     # Fix 1: physically meaningful warm-start - flat smile at median IV
235     # -----
236     # intercept = median mark IV of the calibration sample
237     # all other coefficients = 0
238     # This is a valid point in the price-space landscape (the surface
239     # predicts a constant IV across all strikes and maturities equal to
240     # the sample median) and avoids the IV-space local minimum that the
241     # OLS warm-start introduces.
242     atm_iv = float(np.median(sigma_atm))
243     coefs_init = np.zeros(X.shape[1])
244     coefs_init[0] = atm_iv # intercept only
245
246     if minimize is None:
247         # scipy unavailable - return flat-smile solution
248         model_px_flat = _price_from_coefs(coefs_init)
249         price_rmse = float(np.sqrt(np.mean(
250             ((model_px_flat - market_price) / np.maximum(market_price, 0.5)) ** 2
251         )))
252         return ImpliedSurfaceParams(
253             tuple(float(v) for v in coefs_init),
254             feature_names,
255             sigma_floor=sigma_floor,
256             fit_rmse=price_rmse,
257         )
258
259     # -----
260     # Fix 1 continued: direct price-space optimisation, more iterations
261     # -----
262     # Ridge penalty applied inside the objective via a regularisation term
263     # so the optimiser sees the penalised landscape from the first step.
264     ridge_vec = np.full(X.shape[1], ridge)
265     ridge_vec[0] = 0.0 # do not penalise the intercept
266
267     def objective_regularised(c: np.ndarray) -> float:
268         price_loss = objective(c)
269         reg = float(np.dot(ridge_vec, c ** 2))
270         return price_loss + reg
271
272     result = minimize(
273         objective_regularised,
274         coefs_init,
275         method="L-BFGS-B",
276         options={
277             "maxiter": 2000, # was 400 - needs more iterations from cold start
278             "ftol": 1e-12, # was 1e-10
279             "gtol": 1e-7, # gradient tolerance
280         },
281     )
282     coefs_opt = result.x
283
284     # Final price RMSE (unpenalised, matches summarize_candidate_errors metric)
285     model_px_final = _price_from_coefs(coefs_opt)
286     price_rmse = float(np.sqrt(np.mean(
287         ((model_px_final - market_price) / np.maximum(market_price, 0.5)) ** 2
288     )))
289
290     return ImpliedSurfaceParams(
291         tuple(float(v) for v in coefs_opt),
292         feature_names,
293         sigma_floor=sigma_floor,
294         fit_rmse=price_rmse,
295     )
296
297

```



```

298 def predict_implied_surface_iv(options: pd.DataFrame, params: ImpliedSurfaceParams) -> np.ndarray:
299     """Predict current-regime implied volatility for each option row.
300
301     The adaptive floor replaces the fixed params.sigma_floor for very short
302     maturities ( $T < 7$  days). Near expiry, the polynomial surface can produce
303     implausibly low IV in sparse strike regions; a time-scaled minimum of
304      $\max(\text{sigma\_floor}, 0.15 * \sqrt{30/T\_days})$  prevents those artefacts from
305     inflating pricing error on short-dated options without affecting longer
306     maturities where the surface is well-identified.
307     """
308
309     X, _ = _surface_design(options)
310     X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)
311     sigma = np.einsum("ij,j->i", X, np.asarray(params.coefficients, dtype=float))
312     # Adaptive floor: tighter for short-dated options where the surface
313     # extrapolates poorly, relaxed for longer maturities.
314     T = np.maximum(options["time_to_maturity"].to_numpy(float), 1e-8)
315     T_days = T * 365.25
316     adaptive_floor = np.maximum(
317         params.sigma_floor,
318         0.15 * np.sqrt(np.maximum(30.0 / T_days, 1.0)),
319     )
320     return np.clip(sigma, adaptive_floor, params.sigma_cap)
321
322 def implied_surface_price(
323     options: pd.DataFrame,
324     params: ImpliedSurfaceParams,
325     *,
326     rate_override: float | None = None,
327 ) -> np.ndarray:
328     """Price options by plugging the fitted regime surface into Black-Scholes."""
329
330     sigma = predict_implied_surface_iv(options, params)
331     return np.asarray(
332         bs_price(
333             options["underlying_price"],
334             options["strike"],

```

Listing 11: MR-ISVM 12-factor implied-volatility surface fit and Black-Scholes repricing.

```

344 class RegimeLabel(enum.Enum):
345     """Discrete market regimes used by the multi-regime MR-ISVM surface."""
346
347     CALM = "calm" # low realised vol, no strong directional bias
348     STRESS = "stress" # high realised vol or crash-skew environment
349     TREND = "trend" # intermediate vol with a persistent directional drift
350
351     def __str__(self) -> str:
352         return self.value
353
354 # Regime-classification thresholds (tuned to BTC/ETH empirical distributions).
355 _HIGH_VOL_THRESH : float = 0.65 # annualised realised vol above -> STRESS
356 _CRASH_SKEW_THRESH : float = -0.7 # return skewness below -> STRESS
357 _TREND_DRIFT_THRESH : float = 0.40 # |annualised drift| above -> TREND
358 _MIN_OPTIONS_PER_REGIME: int = 25 # fewer usable options -> skip, use fallback
359
360
361 @dataclass(frozen=True)
362 class MultiRegimeSurfaceParams:
363     """
364     One ImpliedSurfaceParams per detected regime plus the active regime label.
365
366     Attributes
367     -----
368     regime_surfaces : dict[RegimeLabel, ImpliedSurfaceParams]
369         Fitted surface for each regime that had enough options to calibrate.
370         A missing regime falls back to the CALM surface.
371     current_regime : RegimeLabel
372         The regime detected from the most-recent returns at calibration time.
373     fallback_label : RegimeLabel
374         Which surface to use when the requested regime is absent (default CALM).
375     """
376
377     regime_surfaces: dict[RegimeLabel, ImpliedSurfaceParams]

```



```

379     current_regime: RegimeLabel
380     fallback_label: RegimeLabel = RegimeLabel.CALM
381
382     @property
383     def active_params(self) -> ImpliedSurfaceParams:
384         """ImpliedSurfaceParams for the current regime (with CALM fallback)."""
385         return self.regime_surfaces.get(
386             self.current_regime,
387             self.regime_surfaces[self.fallback_label],
388         )
389
390     def params_for(self, label: RegimeLabel) -> ImpliedSurfaceParams:
391         """Return the surface for an explicit regime (with fallback)."""
392         return self.regime_surfaces.get(label, self.active_params)
393
394     @property
395     def n_regimes(self) -> int:
396         """Number of distinct regimes that were successfully calibrated."""
397         return len(self.regime_surfaces)
398
399     def __repr__(self) -> str:
400         labels = ", ".join(r.value for r in self.regime_surfaces)
401         return (
402             f"MultiRegimeSurfaceParams("
403             f"regimes=[{labels}], "
404             f"current={self.current_regime.value}, "
405             f"n_regimes={self.n_regimes})"
406         )
407
408
409     def detect_market_regime(
410         return_frame: pd.DataFrame,
411         *,
412         lookback: int = 20,
413         high_vol_thresh: float = _HIGH_VOL_THRESH,
414         crash_skew_thresh: float = _CRASH_SKEW_THRESH,
415         trend_drift_thresh: float = _TREND_DRIFT_THRESH,
416     ) -> RegimeLabel:
417         """
418         Classify the current market into one of three regimes from recent returns.
419
420         Parameters
421         -----
422         return_frame : pd.DataFrame
423             Must contain a "log_return" column. ``attrs["periods_per_year"]`` is
424             used for annualisation (default 365.25 for crypto).
425         lookback : int
426             Number of most-recent observations used for classification (default 30).
427         high_vol_thresh : float
428             Annualised realised-vol threshold above which the regime is STRESS.
429         crash_skew_thresh : float
430             Return skewness threshold below which the regime is STRESS.
431         trend_drift_thresh : float
432             |Annualised drift| threshold above which the regime is TREND.
433
434         Returns
435         -----
436         RegimeLabel
437             Priority: STRESS > TREND > CALM.
438         """
439         returns = return_frame["log_return"].dropna()
440         if len(returns) < max(lookback // 2, 5):
441             return RegimeLabel.CALM # not enough history - safest default
442
443         ppy = float(return_frame.attrs.get("periods_per_year", 365.25))
444         recent = returns.iloc[-lookback:]
445
446         realized_vol = float(recent.std(ddof=1)) * np.sqrt(ppy)
447         skewness = float(recent.skew())
448         annualized_drift = float(recent.mean()) * ppy
449
450         # STRESS: high vol or crash-type negative skew
451         if realized_vol > high_vol_thresh or skewness < crash_skew_thresh:
452             return RegimeLabel.STRESS
453
454         # TREND: strong directional bias

```

```

455     if abs(annualized_drift) > trend_drift_thresh:
456         return RegimeLabel.TREND
457
458     return RegimeLabel.CALM
459
460
461 def _label_option_regimes(
462     options: pd.DataFrame,
463     return_frame: pd.DataFrame,
464     *,
465     regime_window: int = 30,
466 ) -> pd.Series:
467     """
468     Assign a RegimeLabel to every option row from the returns that preceded it.
469
470     If options have no parseable date information, every row receives the
471     current (most-recent) regime, so the multi-regime fit degrades gracefully
472     to a single-regime calibration.
473
474     Returns
475     -----
476     pd.Series[RegimeLabel] - same index as `options`.
477     """
478     valid_returns = return_frame.dropna(subset=["log_return"]).copy()
479     ppy = float(return_frame.attrs.get("periods_per_year", 365.25))
480
481     # Build a date -> regime mapping over the return time series.
482     if "datetime" in valid_returns.columns:
483         ret_dates = pd.to_datetime(valid_returns["datetime"], utc=True, errors="coerce")
484     elif "timestamp" in valid_returns.columns:
485         ret_dates = pd.to_datetime(valid_returns["timestamp"], unit="ms", utc=True, errors="coerce")
486     elif isinstance(valid_returns.index, pd.DatetimeIndex):
487         ret_dates = pd.Series(pd.to_datetime(valid_returns.index, utc=True), index=valid_returns.index)
488     else:
489         current = detect_market_regime(return_frame)
490         return pd.Series(current, index=options.index)
491
492     date_mask = pd.Series(ret_dates, index=valid_returns.index).notna()
493     valid_returns = valid_returns.loc[date_mask]
494     ret_dates = pd.Series(ret_dates, index=valid_returns.index)
495     if valid_returns.empty:
496         current = detect_market_regime(return_frame)
497         return pd.Series(current, index=options.index)
498
499     date_to_regime: dict[pd.Timestamp, RegimeLabel] = {}
500     log_returns = valid_returns["log_return"].to_numpy(float)
501
502     for i, dt in enumerate(ret_dates):
503         if i < regime_window // 2:
504             label = RegimeLabel.CALM
505         else:
506             window_ret = log_returns[max(0, i - regime_window): i]
507             tmp = pd.DataFrame({"log_return": window_ret})
508             tmp.attrs["periods_per_year"] = ppy
509             label = detect_market_regime(tmp, lookback=len(tmp))
510             date_to_regime[dt] = label
511
512     if not date_to_regime:
513         current = detect_market_regime(return_frame)
514         return pd.Series(current, index=options.index)
515
516     sorted_dates = sorted(date_to_regime.keys())
517
518     current = detect_market_regime(return_frame)
519
520 def _nearest(opt_date: pd.Timestamp) -> RegimeLabel:
521     if pd.isna(opt_date):
522         return current
523     idx = np.searchsorted(sorted_dates, opt_date, side="right") - 1
524     return date_to_regime[sorted_dates[max(idx, 0)]]
525
526 # Try to extract an observation date from the options DataFrame.
527 opt_dates: pd.Series | None = None
528 for col in ("valuation_datetime", "datetime", "date", "timestamp", "valuation_timestamp", "
529             expiration_datetime", "expiry_date"):
530     if col in options.columns:

```

```

530         try:
531             if col.endswith("timestamp") and pd.api.types.is_numeric_dtype(options[col]):
532                 opt_dates = pd.to_datetime(options[col], unit="ms", utc=True, errors="coerce")
533             else:
534                 opt_dates = pd.to_datetime(options[col], utc=True, errors="coerce")
535             break
536         except Exception:
537             pass
538     if opt_dates is None and isinstance(options.index, pd.DatetimeIndex):
539         opt_dates = pd.Series(options.index, index=options.index)
540
541     if opt_dates is None:
542         return pd.Series(current, index=options.index)
543
544     return opt_dates.map(_nearest)
545
546
547 def fit_implied_surface_multi_regime(
548     options: pd.DataFrame,
549     return_frame: pd.DataFrame,
550     *,
551     max_options_per_regime: int = 60,
552     ridge: float = 1e-3,
553     regime_window: int = 30,
554 ) -> MultiRegimeSurfaceParams:
555     """
556     Fit one ImpliedSurfaceParams per detected market regime (multi-regime MR-ISVM).
557
558     This is the regime-aware companion to ``fit_implied_surface``. It:
559
560     1. Detects the current regime via ``detect_market_regime``.
561     2. Labels every option row with the regime that prevailed on its date via
562        ``label_option_regimes``.
563     3. Calls the existing ``fit_implied_surface`` independently for each
564        regime slice that has  $\geq$  ``MIN_OPTIONS_PER_REGIME`` usable rows.
565     4. Guarantees the CALM regime always has a surface (falls back to fitting
566        on all options if the CALM slice is too small).
567
568     Parameters
569     -----
570     options : pd.DataFrame
571         Full option universe for one currency.
572     return_frame : pd.DataFrame
573         Historical log-returns for the same currency.
574     max_options_per_regime : int
575         Forwarded to ``fit_implied_surface`` for each regime slice.
576     ridge : float
577         Ridge penalty forwarded to ``fit_implied_surface``.
578     regime_window : int
579         Rolling lookback (in return observations) for per-date labelling.
580
581     Returns
582     -----
583     MultiRegimeSurfaceParams
584         Call ``.active_params`` to get the ImpliedSurfaceParams for the current
585         regime - fully compatible with ``price_candidate_models``.
586     """
587     current_regime = detect_market_regime(return_frame)
588     regime_labels = _label_option_regimes(options, return_frame, regime_window=regime_window)
589
590     regime_surfaces: dict[RegimeLabel, ImpliedSurfaceParams] = {}
591
592     for regime in RegimeLabel:
593         slice_opts = options[regime_labels == regime].copy()
594         usable = slice_opts.dropna(subset=["mark_iv_decimal"])
595         usable = usable[usable["mark_iv_decimal"].between(0.01, 4.0)]
596
597         if len(usable) < MIN_OPTIONS_PER_REGIME:
598             continue # not enough data; this regime will use the CALM fallback
599
600         try:
601             regime_surfaces[regime] = fit_implied_surface(
602                 slice_opts,
603                 max_options=max_options_per_regime,
604                 ridge=ridge,
605             )

```

```

606         except Exception:
607             continue # degenerate slice - skip silently
608
609         # Always guarantee a CALM surface as the universal fallback.
610         if RegimeLabel.CALM not in regime_surfaces:
611             try:
612                 regime_surfaces[RegimeLabel.CALM] = fit_implied_surface(
613                     options,
614                     max_options=max_options_per_regime,
615                     ridge=ridge,
616                 )
617             except Exception as exc:
618                 raise RuntimeError(
619                     "MR-ISVM multi-regime: could not fit even the fallback CALM surface. "
620                     "Ensure the options DataFrame contains usable mark_iv_decimal values."
621                 ) from exc
622
623         return MultiRegimeSurfaceParams(
624             regime_surfaces=regime_surfaces,
625             current_regime=current_regime,
626             fallback_label=RegimeLabel.CALM,
627         )
628
629
630 def predict_implied_surface_iv_multi_regime(
631     options: pd.DataFrame,
632     params: MultiRegimeSurfaceParams,
633     *,
634     regime_override: RegimeLabel | None = None,
635 ) -> np.ndarray:
636     """
637     Predict IV using the regime-specific surface.
638
639     Delegates to the existing ``predict_implied_surface_iv`` after selecting
640     the correct ImpliedSurfaceParams for the active (or overridden) regime.
641
642     Parameters
643     -----
644     regime_override : RegimeLabel or None
645         Force prediction from a specific regime's surface (e.g. for stress
646         testing). When None (default), the current regime is used.
647     """
648     surface = params.params_for(regime_override) if regime_override is not None else params.active_params
649     return predict_implied_surface_iv(options, surface)
650
651
652 def implied_surface_price_multi_regime(
653     options: pd.DataFrame,
654     params: MultiRegimeSurfaceParams,
655     *,
656     rate_override: float | None = None,
657     regime_override: RegimeLabel | None = None,
658 ) -> np.ndarray:
659     """
660     Price options using the regime-aware MR-ISVM surface.
661
662     Drop-in multi-regime replacement for ``implied_surface_price``. Selects
663     the active regime's ImpliedSurfaceParams then delegates to the existing
664     ``implied_surface_price``, preserving all IV-clamping and adaptive-floor
665     logic exactly.
666
667     Parameters
668     -----
669     rate_override : float or None
670         Overrides the "rate" column for all options when supplied.
671     regime_override : RegimeLabel or None
672         Force a specific regime surface (e.g. for scenario analysis).
673     """
674     surface = params.params_for(regime_override) if regime_override is not None else params.active_params
675     return implied_surface_price(options, surface, rate_override=rate_override)
676
677
678 def summarize_regime_surfaces(params: MultiRegimeSurfaceParams) -> pd.DataFrame:
679     """
680     Tidy DataFrame comparing the fitted surfaces across all calibrated regimes.
681 
```

```

682 Columns: regime, n_coefficients, fit_rmse, sigma_floor, sigma_cap, active.
683 Purely for inspection - no effect on pricing.
684 """
685 rows = []
686 for label, surface in params.regime_surfaces.items():
687     rows.append({
688         "regime": label.value,
689         "n_coefficients": len(surface.coefficients),
690         "fit_rmse": round(surface.fit_rmse, 6),

```

Listing 12: Regime detection and multi-regime MR-ISVM surface wrapper.

```

790 def _variance_state_from_returns(
791     return_frame: pd.DataFrame,
792     merton_params: MertonParams,
793 ) -> SVCJMomentParams:
794     r = return_frame["log_return"].dropna()
795     ppy = float(return_frame.attrs.get("periods_per_year", 365.25))
796     dt_approx = 1.0 / ppy
797
798     rolling_var = r.rolling(20, min_periods=5).var() * ppy
799     rolling_var = rolling_var.dropna()
800
801     theta = float(np.clip(merton_params.sigma ** 2, 1e-6, 9.0))
802     v0 = float(np.clip(
803         rolling_var.iloc[-1] if not rolling_var.empty else theta,
804         0.05 * theta, 9.0,
805     ))
806
807     acf = float(rolling_var.autocorr(lag=1)) if len(rolling_var) > 10 else 0.75
808     kappa_v = float(np.clip(-np.log(np.clip(acf, 0.05, 0.98)) * ppy, 0.25, 12.0))
809
810     jump_returns = return_frame.loc[
811         return_frame.get("is_jump", pd.Series(False, index=return_frame.index)).astype(bool),
812         "log_return",
813     ]
814     if len(jump_returns) > 0:
815         variance_jump_mean = float(np.maximum(
816             jump_returns.pow(2).mean() * ppy - theta,
817             0.05 * theta,
818         ))
819     else:
820         variance_jump_mean = 0.05 * theta
821     variance_jump_mean = float(np.clip(variance_jump_mean, 1e-8, 4.0 * theta))
822
823     if len(rolling_var) > 10:
824         dv = rolling_var.diff().dropna()
825         mean_v = float(rolling_var.mean())
826         denom = np.sqrt(max(mean_v, 1e-8) * dt_approx * 20.0)
827         xi = float(np.clip(dv.std() / denom, 0.05, 3.0))
828     else:
829         xi = 0.5
830
831     variance_change = rolling_var.diff().dropna()
832     aligned_returns = r.reindex(variance_change.index)
833     if (
834         len(variance_change) > 5
835         and aligned_returns.std(ddof=1) > 0
836         and variance_change.std(ddof=1) > 0
837     ):
838         leverage_rho = float(np.corrcorrcoef(aligned_returns, variance_change)[0, 1])
839     else:
840         leverage_rho = -0.5
841
842     return SVCJMomentParams(
843         v0=v0,
844         theta=theta,
845         kappa_v=kappa_v,
846         xi=float(np.clip(xi, 0.05, 3.0)),
847         leverage_rho=float(np.clip(leverage_rho, -0.95, 0.95)),
848         variance_jump_mean=variance_jump_mean,
849         variance_multiplier=1.0,
850         jump_intensity=merton_params.jump_intensity,
851         jump_mean=merton_params.jump_mean,
852         jump_vol=merton_params.jump_vol,
853 )

```

```

854
855 def svcj_average_variance(
856     params: SVCJMomentParams,
857     T: np.ndarray | float,
858 ) -> np.ndarray:
859     """
860     Compute the risk-neutral expected average variance  $E[1/T \int_0^T \sigma^2 dt]$ 
861     for an SVCJ process, using the closed-form moment from the affine
862     Heston + jump framework.
863
864     Formula (Gatheral 2006, Ch. 2 + jump compensator)
865     -----
866     For the CIR variance process with jumps:
867
868          $E[\sigma^2] = \theta + (v_0 - \theta) e^{-\kappa T} + \lambda \mu_v / \kappa (1 - e^{-\kappa T})$ 
869
870     Integrating over  $[0, T]$  and dividing by  $T$ :
871
872          $avg\_var = \theta + (v_0 - \theta) (1 - e^{-\kappa T}) / (\kappa T) + \lambda \mu_v / \kappa [1 - (1 - e^{-\kappa T}) / (\kappa T)]$ 
873
874     The vol-of-vol ( $\xi$ ) and leverage ( $\text{leverage\_rho}$ ) do not enter the
875     *first* moment of the variance, but they do affect the variance of
876     realised variance (the vol-of-vol risk premium). We add a small
877     convexity correction term  $+1/2 \xi^2 \sigma / \kappa$  (derived from the second
878     moment) so that higher  $\xi$  produces a higher effective average variance,
879     consistent with the SVCJ implied-vol surface being wider than Heston.
880
881     Finally, variance_multiplier scales the result (calibrated to prices).
882     """
883     horizon = np.maximum(np.asarray(T, dtype=float), 1e-8)
884     kappa = max(params.kappa_v, 1e-8)
885     lam = max(params.jump_intensity, 0.0)
886     mu_v = max(params.variance_jump_mean, 0.0)
887     xi = params.xi
888     v0 = params.v0
889     theta = params.theta
890
891     # mean-reversion decay averaged over  $[0, T]$ 
892     decay_avg = (1.0 - np.exp(-kappa * horizon)) / (kappa * horizon)
893
894     # CIR mean path (no jumps)
895     base_avg = theta + (v0 - theta) * decay_avg
896
897     # variance-jump contribution to the mean
898     #  $\lambda \mu_v / \kappa * [1 - (1 - e^{-\kappa T}) / (\kappa T)]$ 
899     jump_avg = lam * mu_v / kappa * (1.0 - decay_avg)
900
901     # vol-of-vol convexity correction:  $+1/2 \xi^2 * avg\_var / \kappa$ 
902     # This is the leading term of  $\text{Var}(\text{realised vol})$  that shifts the
903     # risk-neutral expectation of realised variance upward.
904     base_for_correction = np.maximum(base_avg + jump_avg, 1e-10)
905     xi_correction = 0.5 * xi ** 2 * base_for_correction / kappa
906
907     avg_var = (base_avg + jump_avg + xi_correction) * params.variance_multiplier
908     return np.maximum(avg_var, 1e-10)
909
910
911 def svcj_moment_price(
912     options: pd.DataFrame,
913     params: SVCJMomentParams,
914     *,
915     rate_override: float | None = None,
916 ) -> np.ndarray:
917     """
918     Price options with a moment-matched SVCJ approximation.
919
920     The effective diffusive volatility is taken from
921     ``svcj_average_variance`` (which now properly accounts for  $\xi$ , the
922     variance-jump mean, and a convexity correction). Jump risk is then
923     layered on top via the Merton series, using the calibrated jump
924     parameters - exactly the same structure as the original, but now
925     with a better sigma_eff.
926     """
927     avg_var = svcj_average_variance(params, options["time_to_maturity"].to_numpy(float))

```

```

930     sigma_eff = np.sqrt(avg_var)
931     return np.asarray(
932         merton_price(
933             options["underlying_price"],
934             options["strike"],
935             options["time_to_maturity"],
936             _option_rates(options, rate_override),
937             options["option_type"],
938             sigma=sigma_eff,
939             jump_intensity=params.jump_intensity,
940             jump_mean=params.jump_mean,
941             jump_vol=params.jump_vol,
942         )
943     )
944
945
946 def calibrate_svcj_moment_proxy(
947     options: pd.DataFrame,
948     return_frame: pd.DataFrame,
949     merton_params: MertonParams,
950     *,
951     max_options: int = 80,
952 ) -> SVCJMomentParams:
953     """
954     Calibrate the SVCJ moment proxy to option prices.
955
956     The price proxy depends on the average variance path, so the option
957     calibration only refines parameters that enter that path directly:
958     ``variance_multiplier`` and ``xi``. ``leverage_rho`` is still estimated
959     from historical return/variance co-movement, but it is not optimized here
960     because this moment-matched proxy does not use it in the pricing equation.
961     """
962     base = _variance_state_from_returns(return_frame, merton_params)
963     sample = calibration_sample(options, max_options=max_options)
964     if sample.empty or minimize is None:
965         return base
966
967     market = sample["market_price_usd"].to_numpy(float)
968     weights = 1.0 / np.maximum(market, 0.5)
969
970     def objective(x: np.ndarray) -> float:
971         vm, xi = float(x[0]), float(x[1])
972         params = replace(
973             base,
974             variance_multiplier=vm,
975             xi=xi,
976         )
977         model = svcj_moment_price(sample, params)
978         errors = (model - market) / np.maximum(market, 0.5)
979         return float(np.sqrt(np.average(errors ** 2, weights=weights)))
980
981     x0 = np.array([base.variance_multiplier, base.xi], dtype=float)
982     bounds = [(0.25, 4.0), (0.05, 3.0)]
983     result = minimize(
984         objective,
985         x0,
986         method="L-BFGS-B",
987         bounds=bounds,
988         options={"maxiter": 300, "ftol": 1e-8},
989     )
990     vm_opt, xi_opt = (float(v) for v in result.x)
991     return replace(
992         base,
993         variance_multiplier=vm_opt,
994         xi=float(np.clip(xi_opt, 0.05, 3.0)),
995     )

```

Listing 13: SVCJ moment proxy, identifiable calibration, and Merton-layered pricing.

```

998 def _initial_dynamic_jump_params(return_frame: pd.DataFrame, merton_params: MertonParams) -> DynamicJumpParams:
999
1000     jumps = return_frame.get("is_jump", pd.Series(False, index=return_frame.index)).astype(bool)
1001     ppy = float(return_frame.attrs.get("periods_per_year", 365.25))
1002     unconditional = max(float(jumps.mean()), 1.0 / max(len(jumps), 1))
1003     recent_window = max(20, int(round(ppy / 12.0)))
1004     recent = max(float(jumps.tail(recent_window).mean()), unconditional)

```



```

1004     current_scale = np.clip(recent / unconditional, 0.75, 4.0)
1005     return DynamicJumpParams(
1006         sigma=merton_params.sigma,
1007         base_intensity=float(max(1e-4, 0.65 * merton_params.jump_intensity)),
1008         current_intensity=float(max(1e-4, merton_params.jump_intensity * current_scale)),
1009         mean_reversion=6.0,
1010         jump_mean=merton_params.jump_mean,
1011         jump_vol=merton_params.jump_vol,
1012     )
1013
1014
1015 def dynamic_effective_intensity(params: DynamicJumpParams, T: np.ndarray | float) -> np.ndarray:
1016     """Average the mean-reverting intensity over the option horizon."""
1017
1018     horizon = np.maximum(np.asarray(T, dtype=float), 1e-8)
1019     beta = max(params.mean_reversion, 1e-8)
1020     decay_avg = (1.0 - np.exp(-beta * horizon)) / (beta * horizon)
1021     return np.maximum(params.base_intensity + (params.current_intensity - params.base_intensity) * decay_avg,
1022         1e-8)
1023
1024 def dynamic_jump_price(
1025     options: pd.DataFrame,
1026     params: DynamicJumpParams,
1027     *,
1028     rate_override: float | None = None,
1029 ) -> np.ndarray:
1030     """Price options under a Merton mixture with maturity-dependent intensity."""
1031
1032     lambda_eff = dynamic_effective_intensity(params, options["time_to_maturity"].to_numpy(float))
1033     return np.asarray(
1034         merton_price(
1035             options["underlying_price"],
1036             options["strike"],
1037             options["time_to_maturity"],
1038             _option_rates(options, rate_override),
1039             options["option_type"],
1040             sigma=params.sigma,
1041             jump_intensity=lambda_eff,
1042             jump_mean=params.jump_mean,
1043             jump_vol=params.jump_vol,
1044         )
1045     )
1046
1047
1048 def calibrate_dynamic_jump(
1049     options: pd.DataFrame,
1050     return_frame: pd.DataFrame,
1051     merton_params: MertonParams,
1052     *,
1053     max_options: int = 80,
1054 ) -> DynamicJumpParams:
1055     """Calibrate dynamic jump intensity parameters on the same option sample."""
1056
1057     base = _initial_dynamic_jump_params(return_frame, merton_params)
1058     sample = calibration_sample(options, max_options=max_options)
1059     if sample.empty or minimize is None:
1060         return base
1061     market = sample["market_price_usd"].to_numpy(float)
1062
1063     def objective(x: np.ndarray) -> float:
1064         sigma, base_intensity, current_intensity, mean_reversion = x
1065         if current_intensity < base_intensity:
1066             return 1e6 + (base_intensity - current_intensity) ** 2
1067         params = DynamicJumpParams(
1068             sigma=float(sigma),
1069             base_intensity=float(base_intensity),
1070             current_intensity=float(current_intensity),
1071             mean_reversion=float(mean_reversion),
1072             jump_mean=base.jump_mean,
1073             jump_vol=base.jump_vol,
1074         )
1075         model = dynamic_jump_price(sample, params)
1076         errors = (model - market) / np.maximum(market, 1.0)
1077         return float(np.mean(errors * errors))
1078

```



```

1079     x0 = np.array([base.sigma, base.base_intensity, base.current_intensity, base.mean_reversion], dtype=float)
1080     bounds = [(0.03, 3.0), (1e-4, 250.0), (1e-4, 400.0), (0.25, 30.0)]
1081     result = minimize(objective, x0, method="L-BFGS-B", bounds=bounds, options={"maxiter": 250})
1082     return DynamicJumpParams(
1083         sigma=float(result.x[0]),
1084         base_intensity=float(result.x[1]),
1085         current_intensity=float(result.x[2]),
1086         mean_reversion=float(result.x[3]),
1087         jump_mean=base.jump_mean,
1088         jump_vol=base.jump_vol,
1089     )
1090
1091
1092 def price_candidate_models(
1093     options: pd.DataFrame,
1094     *,
1095     historical_sigma: float,
1096     merton_params: MertonParams,
1097     surface_params: ImpliedSurfaceParams,
1098     svcj_params: SVCJMomentParams,
1099     dynamic_params: DynamicJumpParams,
1100     garch_params: GarchParams,
1101 ) -> pd.DataFrame:
1102     """Return long-form model prices and errors for the full option universe."""
1103
1104     model_prices = {
1105         "Black-Scholes": np.asarray(
1106             bs_price(
1107                 options["underlying_price"],
1108                 options["strike"],
1109                 options["time_to_maturity"],
1110                 options["rate"].fillna(0.0),
1111                 historical_sigma,
1112                 options["option_type"],
1113             )
1114         ),
1115         "Merton": np.asarray(
1116             merton_price(
1117                 options["underlying_price"],
1118                 options["strike"],
1119                 options["time_to_maturity"],
1120                 options["rate"].fillna(0.0),
1121                 options["option_type"],
1122                 sigma=merton_params.sigma,
1123                 jump_intensity=merton_params.jump_intensity,
1124                 jump_mean=merton_params.jump_mean,
1125                 jump_vol=merton_params.jump_vol,
1126             )
1127         ),
1128         "SVCJ proxy": svcj_moment_price(options, svcj_params),
1129         "MR-ISVM surface": implied_surface_price(options, surface_params),
1130         "Dynamic jump": dynamic_jump_price(options, dynamic_params),
1131         "GARCH variance": garch_price(options, garch_params),
1132     }
1133
1134     rows = []
1135     base_cols = ["currency", "instrument_name", "option_type", "strike", "time_to_maturity", "log_moneyness"]
1136     market = options["market_price_usd"].to_numpy(float)
1137     for model, prices in model_prices.items():
1138         frame = options[base_cols].copy()
1139         frame["model"] = model
1140         frame["market_price_usd"] = market
1141         frame["model_price_usd"] = prices
1142         frame["error_usd"] = frame["model_price_usd"] - frame["market_price_usd"]
1143         frame["abs_error_usd"] = frame["error_usd"].abs()
1144         rows.append(frame)
1145     return pd.concat(rows, ignore_index=True)
1146
1147
1148 def summarize_candidate_errors(priced: pd.DataFrame) -> pd.DataFrame:
1149     """Compute comparable pricing diagnostics for all candidate models."""
1150
1151     return (
1152         priced.groupby(["currency", "model"], as_index=False)
1153         .agg(
1154             options=("abs_error_usd", "size"),

```

```

1155         bias_usd=("error_usd", "mean"),
1156         mae_usd=("abs_error_usd", "mean"),
1157         median_ae_usd=("abs_error_usd", "median"),
1158         rmse_usd=("error_usd", lambda s: float(np.sqrt(np.mean(np.square(s))))),
1159         p90_ae_usd=("abs_error_usd", lambda s: float(s.quantile(0.90))),
1160     )
1161     .sort_values(["currency", "mae_usd"])
1162     .reset_index(drop=True)
1163 )

```

Listing 14: Dynamic jump-intensity pricing and equal-level candidate error summaries.

```

55 def delta_hedge_short_option(
56     path: Iterable[float],
57     *,
58     K: float,
59     T: float,
60     r: float,
61     sigma: float,
62     option_type: str = "put",
63     model: str = "black_scholes",
64     merton_params: dict[str, float] | None = None,
65 ) -> dict[str, float]:
66     """Simulate discrete delta hedging for a short option.
67
68     The hedge uses only the current spot at each rebalance, then realizes the
69     next price move. The final P&L is hedge portfolio value minus option payoff.
70     """
71
72     spots = np.asarray(list(path), dtype=float)
73     if spots.ndim != 1 or len(spots) < 2:
74         raise ValueError("path must contain at least two spot observations")
75     dt = T / (len(spots) - 1)
76     premium, delta = _model_price_delta(spots[0], K, T, r, sigma, option_type, model, merton_params)
77     cash = premium - delta * spots[0]
78     total_turnover = abs(delta * spots[0])
79
80     for i in range(1, len(spots) - 1):
81         cash += math.exp(r * dt)
82         tau = max(T - i * dt, 1e-8)
83         _, new_delta = _model_price_delta(spots[i], K, tau, r, sigma, option_type, model, merton_params)
84         trade = (new_delta - delta) * spots[i]
85         cash -= trade
86         total_turnover += abs(trade)
87         delta = new_delta
88
89     cash += math.exp(r * dt)
90     final_spot = spots[-1]
91     payoff = max(final_spot - K, 0.0) if option_type.lower().startswith("c") else max(K - final_spot, 0.0)
92     hedge_value = delta * final_spot + cash
93     pnl = hedge_value - payoff
94     return {
95         "initial_spot": float(spots[0]),
96         "final_spot": float(final_spot),
97         "premium": float(premium),
98         "payoff": float(payoff),
99         "hedge_value": float(hedge_value),
100         "pnl": float(pnl),
101         "absolute_pnl": float(abs(pnl)),
102         "turnover": float(total_turnover),
103         "model": model,
104     }
105
106
107 def run_hedge_experiment(
108     paths: np.ndarray,
109     *,
110     K: float,
111     T: float,
112     r: float,
113     sigma: float,
114     option_type: str = "put",
115     merton_params: dict[str, float] | None = None,
116 ) -> pd.DataFrame:
117     """Compare Black-Scholes and Merton hedge errors across simulated paths."""
118

```

```

119 rows = []
120 for path_id, path in enumerate(paths):
121     for model in ("black_scholes", "merton"):
122         rows.append(
123             {
124                 "path_id": path_id,
125                 **delta_hedge_short_option(
126                     path,
127                     K=K,
128                     T=T,
129                     r=r,
130                     sigma=sigma,
131                     option_type=option_type,
132                     model=model,
133                     merton_params=merton_params,
134                 ),
135             }
136         )
137 return pd.DataFrame(rows)

```

Listing 15: Discrete delta-hedging simulation for short options.

```

140 def simulate_jump_hedge_experiment(
141     S0: float,
142     *,
143     K: float,
144     T: float,
145     r: float,
146     sigma: float,
147     jump_intensity: float,
148     jump_mean: float,
149     jump_vol: float,
150     steps: int = 60,
151     paths: int = 250,
152     seed: int = 4331,
153     option_type: str = "put",
154 ) -> pd.DataFrame:
155     """Simulate jump paths and compare hedge error distributions."""
156
157     simulated = simulate_merton_paths(
158         S0,
159         T,
160         r=r,
161         sigma=sigma,
162         jump_intensity=jump_intensity,
163         jump_mean=jump_mean,
164         jump_vol=jump_vol,
165         steps=steps,
166         paths=paths,
167         seed=seed,
168     )
169     return run_hedge_experiment(
170         simulated,
171         K=K,
172         T=T,
173         r=r,
174         sigma=sigma,
175         option_type=option_type,
176         merton_params={
177             "jump_intensity": jump_intensity,
178             "jump_mean": jump_mean,
179             "jump_vol": jump_vol,
180         },
181     )

```

Listing 16: Jump-path hedge experiment comparing Black-Scholes and Merton hedges.

```

380 def _candidate_price_scalar(
381     model: str,
382     contract: pd.DataFrame,
383     *,
384     historical_sigma: float,
385     merton_params,
386     surface_params,
387     svcj_params,

```

```

388     dynamic_params,
389     garch_params,
390     vol_bump: float = 0.0,
391 ) -> float:
392     """Price one diagnostic option under a candidate model.
393
394     ``vol_bump`` is a common parallel volatility bump used only for the
395     finite-difference Vega diagnostic. It bumps the model's effective
396     diffusive volatility or fitted IV input while keeping jump parameters fixed.
397     """
398
399     T = contract["time_to_maturity"].to_numpy(float)
400     rate = contract["rate"].fillna(0.0)
401     option_type = contract["option_type"]
402     spot = contract["underlying_price"]
403     strike = contract["strike"]
404
405     if model == "Black-Scholes":
406         sigma = max(historical_sigma + vol_bump, 1e-8)
407         return _scalar(bs_price(spot, strike, T, rate, sigma, option_type))
408     if model == "Merton":
409         return _scalar(
410             merton_price(
411                 spot,
412                 strike,
413                 T,
414                 rate,
415                 option_type,
416                 sigma=max(merton_params.sigma + vol_bump, 1e-8),
417                 jump_intensity=merton_params.jump_intensity,
418                 jump_mean=merton_params.jump_mean,
419                 jump_vol=merton_params.jump_vol,
420             )
421         )
422     if model == "SVCJ proxy":
423         if abs(vol_bump) <= 1e-14:
424             return _scalar(svcj_moment_price(contract, svcj_params))
425         sigma_eff = np.sqrt(svcj_average_variance(svcj_params, T)) + vol_bump
426         return _scalar(
427             merton_price(
428                 spot,
429                 strike,
430                 T,
431                 rate,
432                 option_type,
433                 sigma=np.maximum(sigma_eff, 1e-8),
434                 jump_intensity=svcj_params.jump_intensity,
435                 jump_mean=svcj_params.jump_mean,
436                 jump_vol=svcj_params.jump_vol,
437             )
438         )
439     if model == "MR-ISVM surface":
440         if abs(vol_bump) <= 1e-14:
441             return _scalar(implied_surface_price(contract, surface_params))
442         sigma_iv = np.maximum(predict_implied_surface_iv(contract, surface_params) + vol_bump, 1e-8)
443         return _scalar(bs_price(spot, strike, T, rate, sigma_iv, option_type))
444     if model == "Dynamic jump":
445         bumped = replace(dynamic_params, sigma=max(dynamic_params.sigma + vol_bump, 1e-8))
446         return _scalar(dynamic_jump_price(contract, bumped))
447     if model == "GARCH variance":
448         if abs(vol_bump) <= 1e-14:
449             return _scalar(garch_price(contract, garch_params))
450         sigma_eff = np.sqrt(garch_average_variance(garch_params, T)) + vol_bump
451         return _scalar(bs_price(spot, strike, T, rate, np.maximum(sigma_eff, 1e-8), option_type))
452     raise ValueError(f"Unknown candidate model: {model}")
453
454
455 def _candidate_local_greeks(
456     model: str,
457     currency: str,
458     spot: float,
459     strike: float,
460     T: float,
461     rate: float,
462     option_type: str,
463     **price_kwargs,

```

```

464 ) -> dict[str, float]:
465     contract = _contract_frame(currency, spot, strike, T, rate, option_type)
466     base = _candidate_price_scalar(model, contract, **price_kwargs)
467
468     h_s = max(1e-3, 1e-4 * spot)
469     up_contract = _contract_frame(currency, spot + h_s, strike, T, rate, option_type)
470     dn_contract = _contract_frame(currency, max(spot - h_s, 1e-8), strike, T, rate, option_type)
471     up_s = _candidate_price_scalar(model, up_contract, **price_kwargs)
472     dn_s = _candidate_price_scalar(model, dn_contract, **price_kwargs)
473
474     h_vol = 1e-4
475     up_v = _candidate_price_scalar(model, contract, vol_bump=h_vol, **price_kwargs)
476     dn_v = _candidate_price_scalar(model, contract, vol_bump=-h_vol, **price_kwargs)
477
478     h_r = 1e-4
479     up_r_contract = _contract_frame(currency, spot, strike, T, rate + h_r, option_type)
480     dn_r_contract = _contract_frame(currency, spot, strike, T, rate - h_r, option_type)
481     up_r = _candidate_price_scalar(model, up_r_contract, **price_kwargs)
482     dn_r = _candidate_price_scalar(model, dn_r_contract, **price_kwargs)
483
484     return {
485         "Price USD": base,
486         "Delta": (up_s - dn_s) / (2.0 * h_s),
487         "Gamma": (up_s - 2.0 * base + dn_s) / (h_s * h_s),
488         "Parallel vol Vega": (up_v - dn_v) / (2.0 * h_vol),
489         "Rho": (up_r - dn_r) / (2.0 * h_r),
490     }
491
492
493 def _candidate_price_delta(
494     model: str,
495     currency: str,
496     spot: float,
497     strike: float,
498     T: float,
499     rate: float,
500     option_type: str,
501     **price_kwargs,
502 ) -> tuple[float, float]:
503     contract = _contract_frame(currency, spot, strike, T, rate, option_type)
504     base = _candidate_price_scalar(model, contract, **price_kwargs)
505     h_s = max(1e-3, 1e-4 * spot)
506     up_contract = _contract_frame(currency, spot + h_s, strike, T, rate, option_type)
507     dn_contract = _contract_frame(currency, max(spot - h_s, 1e-8), strike, T, rate, option_type)
508     up_s = _candidate_price_scalar(model, up_contract, **price_kwargs)
509     dn_s = _candidate_price_scalar(model, dn_contract, **price_kwargs)
510     return base, (up_s - dn_s) / (2.0 * h_s)
511
512
513 def _candidate_delta_hedge_path(
514     model: str,
515     path: np.ndarray,
516     *,
517     currency: str,
518     strike: float,
519     T: float,
520     rate: float,
521     option_type: str,
522     price_kwargs: dict,
523 ) -> dict[str, float]:
524     spots = np.asarray(path, dtype=float)
525     dt = T / (len(spots) - 1)
526     premium, delta = _candidate_price_delta(
527         model,
528         currency,
529         float(spots[0]),
530         strike,
531         T,
532         rate,
533         option_type,
534         **price_kwargs,
535     )
536     premium = float(premium)
537     delta = float(delta)
538     cash = premium - delta * spots[0]
539     turnover = abs(delta * spots[0])

```

```

540
541     for i in range(1, len(spots) - 1):
542         cash *= np.exp(rate * dt)
543         tau = max(T - i * dt, 1e-8)
544         _, new_delta = _candidate_price_delta(
545             model,
546             currency,
547             float(spots[i]),
548             strike,
549             tau,
550             rate,
551             option_type,
552             **price_kwargs,
553         )
554         new_delta = float(new_delta)
555         trade = (new_delta - delta) * spots[i]
556         cash -= trade
557         turnover += abs(trade)
558         delta = new_delta
559
560     cash *= np.exp(rate * dt)
561     final_spot = float(spots[-1])
562     payoff = max(final_spot - strike, 0.0) if option_type.lower().startswith("c") else max(strike - final_spot,
563         0.0)
564     hedge_value = delta * final_spot + cash
565     pnl = hedge_value - payoff
566     return {
567         "premium": premium,
568         "payoff": payoff,
569         "hedge_value": float(hedge_value),
570         "pnl": float(pnl),
571         "absolute_pnl": float(abs(pnl)),
572         "turnover": float(turnover),
573     }

```

Listing 17: All-candidate diagnostic pricing, finite-difference Greeks, and hedge-path logic used in the report comparison.

C AI Assistance Statement

AI assistance was used to structure the project, implement Python modules, draft explanatory text, generate plots and tables, and format this report. The quantitative formulas, course alignment, market data, statistical tests, and validation checks are explicit in the codebase and report. The implementation was verified with unit tests and live Deribit data smoke tests.